

Generación de casos y datos de prueba

Contenido de un caso de prueba

Cada caso de prueba consiste en tres valores: **acciones**, **datos** y **resultados esperados**.

Ejemplo: Llamar a Anita por celular

Valores de prueba	Acciones	Resultado esperado
Mi celular: 011-5121314	1. Habilitar acceso 2. Presionar ícono de teléfonos 3. Presionar ícono de búsqueda 4. Buscar número de Anita 5. Seleccionarlo, presionando para llamar 6. Esperar contestación	Celular de Anita Anita: "Hola!"

- Los casos de prueba deben ser finitos, cuantos menos y mejor diseñados, es mejor
- Los casos de prueba buscan errores, queremos el menor número de casos de prueba que encuentre la cantidad máxima de errores.

Armado de los casos de prueba: información a completar:

Los datos necesarios para identificar y armar cada caso de prueba incluyen:

- * **Identificador:** Identificador único para el caso de prueba Es el código con el objeto de vincular, por ejemplo, un informe de errores al caso de prueba en el cual ha sido detectado
- * **Conjunto de prueba:** Identificador de los conjuntos de prueba donde el caso de prueba es usado
- * **Nombre de la Funcionalidad:** Funcionalidad del IP-Inventario de Prueba que se prueba con este caso de prueba
- * **Prioridad:** Prioridad asignada al caso de prueba, por ejemplo: 1- alta, 2- media, 3-baja
- * **Requisitos:** características del objeto de pruebas que el caso de prueba debe evaluar.
- * **Necesidades de ambientes:** si bien no es un requisito obligatorio, debe saberse e indicarse en qué ambiente se van a correr las pruebas. Lo ideal es que haya un ambiente separado de Testing y exclusivo para correr las pruebas, para no ser afectado por que se esté actualizando el código, como ser en el ambiente de desarrollo o por que estén realizando otras pruebas como puede ser en los ambientes de pre-producción donde el cliente prueba para dar su aceptación o en un ambiente de pruebas de automatización donde se pueden afectar los resultados por el uso de dicho ambiente.
 - o **Situación:** puede haber un caso en el que esté probando una aplicación, un desarrollador esté realizando modificaciones en paralelo a la misma aplicación o alguien pueda actualizar la aplicación después de que termine la prueba. Esto conduce a una situación en la que los resultados de su prueba pueden variar con el tiempo.
Por lo tanto, siempre es mejor agregar una marca de tiempo con el nombre del evaluador en los comentarios de la prueba para que el resultado de una prueba (aprobado o reprobado) se pueda atribuir al estado de una aplicación en ese momento en particular.

Alternativamente, puede tener un '**Fecha de ejecución**' Columna agregada por separado al caso de prueba que identificará explícitamente la marca de tiempo de la prueba.

- * **HW:** Lista de Hardware o dispositivos requerido para ejecutar el caso de prueba
- * **SW:** Lista de Software instalado requerido para ejecutar el caso de prueba
- * **Configuración** (setup): Lista de pasos necesarios para comenzar las pruebas
- * **Retroceder acciones** (cleanup): Lista de pasos necesarios para retroceder al estado previo a la prueba.
- * **Precondiciones:** situación inicial o previa a la ejecución de pruebas o características de un Explicación de la acción que se hará con los datos de la situación inicial. Por ejemplo: dar de alta un elemento, agregar un elemento a una lista, ecaso de prueba
- * **Resultado esperado:** Especificación del resultado esperado de ejecutar las pruebas.
- * **Dependencias** (si hay): orden de ejecución de casos de prueba, razón de las dependencias
- * **Valores de entrada:** descripción de los datos de entrada de un objeto de pruebas. Puede ser un dibujo o un texto explicando los datos con los que se probará el caso. Debe ser claro y no dejar dudas.

Los siguientes datos se completan una vez que el caso de prueba se ejecutó:

- * **Resultado real:** Especificación de lo que realmente ocurrió: Pasó/Falló/No se pudo ejecutar
- * **Postcaso de prueba condiciones:** características de un objeto de prueba tras la ejecución de pruebas, descripción de su situación tras la ejecución de las pruebas. Puede ser un dibujo o texto explicando cómo se espera que los datos iniciales queden luego de ejecutar la acción. Debe ser claro y no dejar dudas.

Ejemplo

- *Identificador:* 1
- *Nombre:* Alta de elemento al final de una lista ordenada
- *Situación inicial:* Lista:(3 → 5 → 7 → 15)
- *Acción:* Agregar a la lista el elemento 18
- *Resultado esperado:* Lista: (3 → 5 → 7 → 15 → 18)

- * **Duración:** Tiempo reloj en ejecutar el caso de prueba
- * **Esfuerzo:** Horas-Persona requeridas para configurar y ejecutar el caso de prueba
- * **Fecha:** Fecha en la que se realizó la prueba
- * **Identificación del Defecto:** En caso de que la prueba falle se debe identificar el incidente reportado para su seguimiento
- * **Versión:** Identificador de la versión probada
- * **Tester:** Nombre del tester que ejecutó el caso de prueba

- * **EVIDENCIAS:** son los resultados que se adjuntan con foto/s de la/s pantallas con los resultados (screenshot) y pueden ser también con los pasos realizados. También se pueden realizar videos para mostrar el paso a paso y el resultado final.
 - Si tienes Windows 10: encontrarás las capturas de pantalla dentro de la carpeta Imágenes de tu carpeta de usuario. Es decir, que si vas a **Biblioteca <Imágenes> Capturas de pantalla** encontrarás las capturas hechas con este atajo de teclado. Cada vez que pulses **tecla Windows + Impr Pant** harás una captura completa, de toda la pantalla. Y, por defecto, las capturas se guardan en **formato imagen PNG**, el ideal para este tipo de imágenes.
 - * Otra opción es usar la herramienta Recortes, una de las que hay por defecto en Windows 10 y que permite hacer capturas personalizadas.
 - Pulsa las **teclas Windows + Mayúsculas + S**.
 - Con **ALT + M** eliges un modo de recorte.
 - Con **ALT + N** se crea una nueva captura.
 - Con **CTRL + S** para guardarla
 - Una vez realizada la captura, podrás editarla **escribiendo encima** empleando las herramientas disponibles. Y, al terminar, podrás guardarla o compartirla. También puedes editarla con otras aplicaciones de **imagen o retoque fotográfico**.
-

VIDEOS

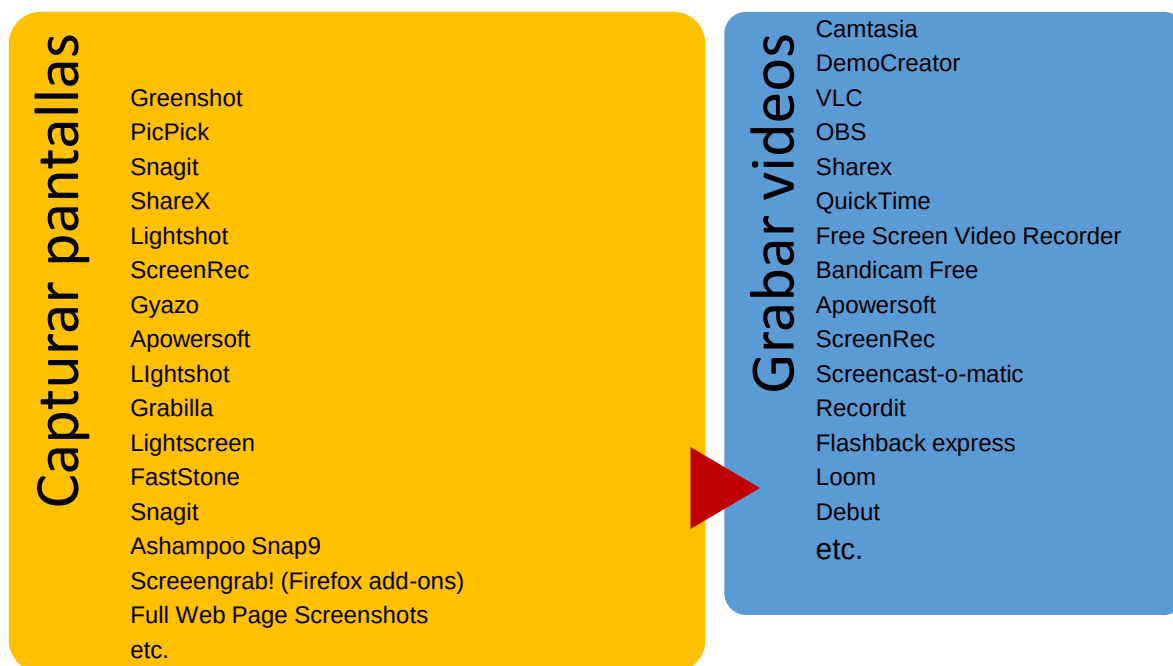
Para grabar la pantalla de un PC con Windows 10 realiza los siguientes pasos:

1. Entra en la aplicación que quieres capturar
2. Escribe Xbox en la caja de búsqueda o pulsa la combinación de teclas Windows +G
3. Activa el audio para grabar tu voz, y empieza a grabar pulsando Grabar o con las teclas [Windows]+[Alt]+[R]
4. Explica los pasos que vas realizando para correr la prueba si hace falta
5. Cuando quieras finalizar la grabación, vuelve a pulsar [Windows]+[Alt]+[R].

Tu vídeo quedará guardado dentro de la carpeta de videos > Capturas en formato .mp4

Herramientas para ayudar a obtener las evidencias

Tener en cuenta: la latencia depende de la velocidad de tu conexión a Internet y por tanto sin un mínimo tendrás siempre esta latencia. Intenta grabar lo más cerca del router si estás usando Wifi. Algunas herramientas son add-ons del browser que estés usando.



Lista de verificación para realizar pruebas

La lista de verificación **es una lista de cosas a verificar antes de decir que la tarea se ha completado o está cubierta**. Del mismo modo en las pruebas de software, la lista de verificación es un documento que cubre la lista de cosas para verificar antes de aprobar la compilación.

La siguiente son listas de verificación que puedes realizar:

1. **Lista de verificación BVT** (Pruebas de verificación de compilación): funcionalidades de alto nivel de la aplicación
2. **Lista de verificación de funcionalidad**: cubre la funcionalidad completa de la aplicación
3. **Lista de verificación de interrupciones**
 - * Cuáles son los mejores métodos para la estimación de proyectos de software?
 - * Cuál es el mejor lenguaje de programación para proyectos de software muy grandes?
4. **Requisitos de certificación**: cubre las pautas de terceros para Play Store y App Store
5. **Lista de verificación de notificaciones push**
6. **Lista de verificación de actualización de funciones**
7. **Lista de verificación específica del dispositivo**
8. **Lista de verificación de localización**
9. **Lista de verificación de compras de aplicaciones**

Mantenga dos hojas separadas: 'Errores' y 'Resumen' en el documento

Si está documentando en Excel, las dos primeras hojas del libro de trabajo deberían ser Resumen y Errores.

- * La hoja **Resumen** debe resumir el escenario de prueba
- * La hoja **Errores** debe enumerar todos los problemas encontrados durante la prueba.

La importancia de agregar estas dos hojas es que brindará una comprensión clara de las pruebas al lector / usuario del documento. Por lo tanto, cuando el tiempo es limitado, estas dos hojas pueden resultar muy útiles para proporcionar una descripción general de las pruebas.

El documento de prueba debe proporcionar la mejor cobertura de prueba posible, una excelente legibilidad y debe seguir un formato estándar en todo momento.

Podemos lograr la excelencia en la documentación de prueba con solo tener en cuenta algunos consejos esenciales como la organización del documento de caso de prueba, priorizar los casos de prueba, tener todo en la secuencia adecuada, incluidos todos los detalles obligatorios para ejecutar un caso de prueba, y proporcionar pasos de prueba claros y lúcidos, etc.

Ejecución de los casos de prueba (corrida)

Cada vez que se ejecuta un caso debe registrarse la información resultante para que quien sea responsable del código probado, sepa si está correcto o se produjo algún error o resultado no deseado.

Es en este paso donde se obtiene un resultado real de la ejecución del código que o denominamos **Resultado Final de la Ejecución del Prueba.**

Las ejecuciones de casos se agrupan en “Corridas” que se deben identificar para saber cuándo (y a veces quién) realizó una ejecución.

Cómo escribir casos de prueba de software

La escritura de casos de prueba varía según lo que el caso de prueba esté midiendo o probando. Esta es también una situación en la que compartir activos de prueba entre los equipos de desarrollo y de prueba puede acelerar las pruebas de software. Pero todo comienza con saber cómo escribir un caso de prueba de manera efectiva y eficiente.

Los casos de prueba tienen algunas partes integrales que siempre deben estar presentes en los campos. Sin embargo, **cada caso de prueba se puede dividir en 8 pasos básicos.**

Paso 1: ID de caso de prueba

Todos los casos de prueba deben llevar ID únicos para representarlos. En la mayoría de los casos, seguir una convención para este ID de nomenclatura ayuda con la organización, la claridad y la comprensión.

Paso 2: Descripción de la prueba

Esta descripción debe detallar qué unidad, característica o función se está probando o qué se está verificando.

Paso 3: Supuestos y condiciones previas

Esto implica que se cumplan las condiciones antes de la ejecución del caso de prueba. Un ejemplo sería requerir una cuenta de Outlook, Gmail, etc. válida para iniciar sesión.

Paso 4: Datos de prueba

Esto se relaciona con las variables y sus valores en el caso de prueba En el ejemplo de un inicio de sesión por correo electrónico, sería el nombre de usuario y la contraseña de la cuenta.

Paso 5: Pasos a ejecutar

Estos deben ser pasos fácilmente repetibles ejecutados desde la perspectiva del usuario final.

Ejemplo, un caso de prueba para iniciar sesión en un servidor de correo electrónico podría incluir estos pasos:

1. Abra la página web del servidor de correo electrónico.
2. Introduzca su nombre de usuario.
3. Introducir la contraseña.
4. Haga clic en el botón "Entrar" o "Iniciar sesión".

Paso 6: Resultado Esperado

Esto indica el resultado esperado después de la ejecución del paso del caso de prueba Al ingresar la información de inicio de sesión correcta, el resultado esperado sería un inicio de sesión exitoso.

Paso 7: Resultado real y condiciones posteriores

En comparación con el resultado esperado, podemos determinar el estado del caso de prueba En el caso del inicio de sesión por correo electrónico, el usuario iniciará sesión correctamente o no. La condición posterior es lo que sucede como resultado de la ejecución del paso, como ser redirigido a la bandeja de entrada del correo electrónico.

Paso 8: Contraseña errónea

La determinación del estado de aprobado / reprobado depende de cómo se comparan entre sí el resultado esperado y el resultado real.

Mismo resultado = Aprobado Diferentes resultados = Falla

Cómo NO escribir pruebas

Pasamos la mayor parte de nuestro tiempo escribiendo, revisando, ejecutando o manteniendo estos. Es bastante lamentable que las pruebas también sean las más propensas a errores. Las diferencias en la comprensión, las prácticas de prueba de la organización, la falta de tiempo, etc. son algunas de las razones por las que a menudo vemos pruebas que dejan mucho que desear.

Escenario de prueba de muestra

Hay que considerar que se deben identificar todos los escenarios (camino) a probar de un caso de uso: flujo básico, sub-flujos y flujos alternativos.

- * Un escenario de prueba (test scenario: TS) **consiste en un procedimiento de prueba detallado.**

- * También podemos decir que un escenario de prueba tiene muchos casos de prueba asociados (1 TS: n TC).
- * Antes de ejecutar el escenario de prueba, debemos pensar en casos de prueba para cada escenario.
- * Son la clasificación de alto nivel de los requisitos de prueba agrupados en función de la funcionalidad de un módulo y puede derivarse de casos de uso o historias de usuarios.
- * El escenario de prueba es una declaración de línea que nos dice qué probar.
- * Test Scenario significa hablar y pensar los requisitos en detalle.
- * Los escenarios de prueba son más importantes cuando el tiempo para escribir casos de prueba no es suficiente y los miembros del equipo están de acuerdo con el escenario detallado.
- * Los buenos escenarios de prueba reducen la complejidad y la repetibilidad del producto.
- * Si el escenario de prueba no es lo suficientemente detallado, entonces podría llevar algún tiempo debatir y comprender de qué escenario de prueba está hablando exactamente.

Ejemplo: un escenario de prueba se puede describir en casos de prueba de la siguiente manera:

- Caso de prueba 1: verifique la operación al ingresar un nombre de usuario y contraseña válidos.
- 2: Verifique la operación si se ingresa un nombre de usuario y contraseña no válidos.
- 3: Verifique la operación si el nombre de usuario está vacío y se presiona el botón de inicio de sesión.
- Y así sucesivamente.

Este tipo de escenarios de prueba son bastante vagos y cubren una amplia variedad de variables. Sin embargo, las pruebas se tratan de ser muy específicas. Es por eso que necesitamos casos de prueba detallados.

Cuándo conviene usar escenarios?

Cuando las reglas de negocio son complejas	Cuando el sistema cambia de un momento a otro	Cuando no se puede reutilizar los casos de prueba	Cuando las diferentes combinaciones para probar una determinada aplicación son muchas
• Es más fácil pensar en el resultado esperando a un determinado flujo de acciones, que tener que escribir paso a paso y además es más efectivo en cuestiones de tiempo.	• Mantener los test cases involucra tiempo, durante el cual a veces esos mismos caso de pruebas ya no útiles para el estado actual de la aplicación.	• Cuando se tiene por ejemplo una aplicación que interactúa con otras aplicaciones pero los pasos para ejecutar determinados casos de prueba no son los mismos, lo único que es igual es el resultado final.	• En este caso escribir la cantidad de casos de prueba para probar una sola parte de la aplicación involucra mucho tiempo y nivel de detalle para diferenciar un test cases de otro.

Consideraciones respecto de las pruebas

El fundamental respeto a la Prueba de Software es que no se puede probar completamente un sistema compuesto por varios programas, por lo que en el momento de realizar las pruebas se deben tomar decisiones respecto a cómo se van a diseñar los casos de prueba. Otro punto importante a tener en cuenta es la actitud que debe tener la persona que realiza las pruebas.

La prueba exhaustiva requiere probar el comportamiento de un programa para todas las combinaciones validas e invalidas de entradas. Además, se debe probar esto en cada punto donde se ingresan datos, bajo cada estado posible del programa en ese punto. Esto no es posible, ya que el tiempo requerido y el costo es prohibitivo.

En un mundo ideal, desearíamos probar cada permutación posible de un programa. Incluso un programa aparentemente simple puede tener centenares o millares de combinaciones posibles de la entrada y de la salida. Crear los casos de prueba para todas estas posibilidades no es económicamente factible. Este problema fundamental, tendrá implicancias para la economía de las pruebas, suposiciones que el tester tendrá que hacer sobre el programa, y la manera en el cual los casos de prueba se diseñan.

El objetivo debe ser maximizar la producción de las pruebas, esto es, maximizar el número de los errores encontrados por un número finito de los casos de prueba.

La forma en cómo se seleccionan los casos de prueba es una de las principales decisiones a tomar, teniendo en cuenta las siguientes consideraciones:

- Los casos de prueba deben ser revisados regularmente, escribiendo nuevas pruebas que ejerciten diferentes partes del software, con el objetivo de encontrar más defectos

Otro aspecto que se debe considerar al realizar pruebas es decidir cuándo el programa falla para un conjunto de datos de entrada, o sea, conocer cuál es la salida esperada del programa. Este problema es conocido como **el problema del oráculo**.

Un oráculo es cualquier agente (humano o mecánico) que decide si un programa se comportó correctamente en una prueba dada, y produce por consiguiente un veredicto de "pasó" (pass) o de "falló" (fail).

Un oráculos es lo que nos dice qué resultado debemos esperar ante determinadas entrada y condiciones de ejecución.

Básicamente es el mecanismo, ya sea manual o automático, de **verificar si el comportamiento del sistema es el deseado o no**. Para esto, el oráculo deberá comparar el valor esperado contra valor obtenido, el estado final esperado con el estado final alcanzado, el tiempo de respuesta aceptable con el tiempo de respuesta obtenido, etc.

Pueden ser:

- personas
- modelos
- Documentos

- productos similares
- normas o estándares

Existen diversas clases de oráculos, y la automatización del oráculo puede llegar a ser muy compleja y costosa. El oráculo más común es el oráculo entrada/salida, que especifica la salida esperada para una entrada específica. Si el programa hace exactamente lo que su especificación dice que debe hacer y no hace nada más, entonces cumple con su especificación.

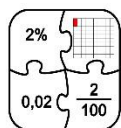
No es razonable decir que un programa es correcto si cumple con su especificación, ya que las especificaciones pueden tener errores.

En este punto es interesante reflexionar sobre algo a lo que se llama la Paradoja del Pesticida: los insectos (bugs, refiriéndose a fallos) que sobreviven a un pesticida se hacen más fuertes, y resistentes a ese pesticida. O sea, **si diseñamos un conjunto de pruebas probablemente ciertos bugs sobrevivan**. Si luego diseñamos una técnica más completa y llamémosle “exhaustiva”, entonces encontraremos más bugs, pero otros seguirán sobreviviendo. Al fin de cuentas **los que van quedando son los más duros de matar, y se van haciendo resistentes a los distintos pesticidas**.

Obtención de casos de prueba a partir de requisitos

- El diseño de casos de prueba debe ser un proceso controlado.
- Los casos de prueba pueden ser creados **formal** o **informalmente**, dependiendo de las características del proyecto y la madurez del proceso en uso.

Cobertura de Pruebas



Es una medida de calidad de las pruebas. Se definen cierto tipo de entidades sobre el sistema, y luego se intenta cubrirlas con las pruebas. **Es una forma de indicar cuándo probamos suficiente, o para tomar ideas de qué otra cosa probar** (pensando en aumentar la cobertura elegida).

Trazabilidad

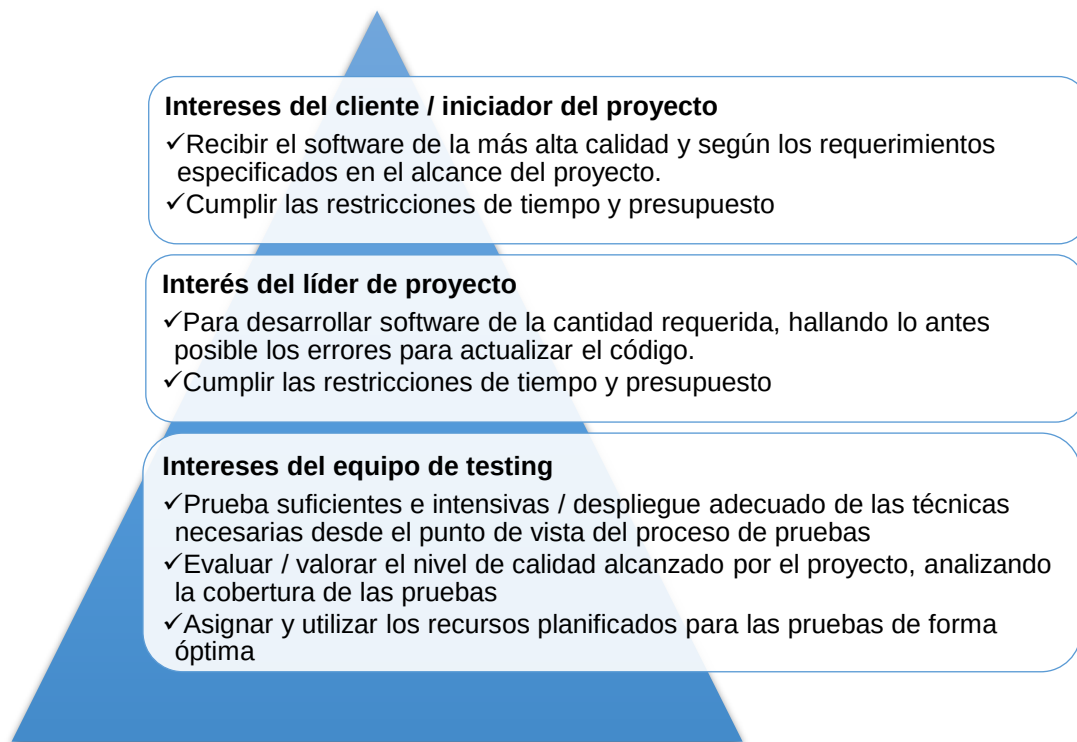
Las pruebas deben ser trazables: ¿qué casos de prueba han sido incluido en el catálogo o listado de pruebas, basados en qué requisitos?

Las consecuencias de los cambios en los requisitos sobre las pruebas a realizar pueden ser identificadas directamente.

La trazabilidad también ayuda a determinar la cobertura de requisitos.

Intereses distintos causan enfoques diferentes en el diseño de pruebas

Se deben aplicar criterios adicionales dependiendo de la situación, tipo de negocio, objetivos del proyecto, alcances del testing, equipo de trabajo, etc.



¿Cuándo tenemos que empezar a construir casos de prueba?

Lo antes posible, empezando la planificación de un “testing básico” generando casos de prueba junto con la especificación de requerimientos. Esto se debe a que los casos de prueba NO deben estar influenciados por las estrategias de solución que hemos de elegir sino por la especificación del problema.

Que casos debo testear?

Idealmente uno querría probar absolutamente todas las posibles combinaciones de datos y situaciones. Esto se hace normalmente imposible debido a la cantidad de casos que deberíamos generar. Lo que se hace comúnmente es reconocer las situaciones que creemos son más extremas y algunas de las más comunes. Este proceso debe ser planificado y documentado.

Especificación de Casos de prueba

Se han desarrollado muchas maneras de escribir y especificar estos Casos. Aquí presentaremos una adaptación que busca simplificar las pruebas:

Ejemplo: si quiero probar si mi solución de agregar un elemento a una lista simplemente vinculada que está ordenada podría plantear las siguientes situaciones o casos (estos son sólo algunos ejemplos):

- Agregar un elemento a una lista vacía.
- Agregar un elemento a la lista que tiene un solo elemento.
- Agregar un elemento al principio de la lista.
- Agregar un elemento al medio de la lista.
- Agregar un elemento al final de la lista.
- Agregar un elemento que ya existe en la lista.

Construcción de Casos de Prueba: Este paso debe realizarse muy temprano en el proceso de desarrollo, tal como se mencionó anteriormente. El resultado de este paso es una especificación de Casos de Prueba, que pueden tener una estructura como la que proponemos en el punto siguiente (*“Estructura de los Casos de Prueba”*).

Ejecución de los Casos de Prueba: (*comúnmente llamada “corrida”*) La ejecución de los casos se realiza una vez terminada la codificación. Esta ejecución consiste en tomar cada especificación de caso de prueba, ejecutar el software que estamos probamos (puede ser una porción de código) y registrar el resultado de la ejecución. Normalmente una corrida implica la ejecución de TODOS los casos de prueba. Un mismo Caso de Prueba se puede ejecutar muchas veces, tantas como corridas se realicen, idealmente al menos cada vez que se modifica algo del código.

Los 3 problemas más comunes en casos de prueba

1. Pasos compuestos

En primer lugar, **¿qué es un paso compuesto? Es aquel que se puede dividir en varios pasos individuales.**

Ejemplo: estás dando instrucciones desde el punto A al punto B: si dices que 'Ve al lugar XYZ y luego al ABC', esto no tendrá mucho sentido, porque aquí nos encontramos pensando: '¿Cómo puedo llegar a XYZ en primer lugar'- en lugar de comenzar con “Gire a la izquierda desde aquí y avance 1 kilómetro, luego gire a la derecha en R número 11 para llegar a XYZ ” podría lograr mejores resultados.

Las mismas reglas exactas se aplican a las pruebas y sus pasos también.

Por ejemplo: escribiendo una prueba para Amazon.com: haga un pedido de cualquier producto. Recuerde, las pruebas siempre tratan sobre 'Cómo' realizar la prueba, por lo que es importante escribir los pasos exactos de 'Cómo pagar y pagar' en su prueba.

- a. Lanzar Amazon.com
- b. Busque un producto ingresando la palabra clave / nombre del producto en el campo 'Buscar' en la parte superior de la pantalla.
- c. De los resultados de búsqueda que se muestran, elija el primero.
- d. Haga clic en Agregar al carrito en la página de detalles del producto.
- e. Checkout y pago.
- f. Consulta la página de confirmación del pedido.

El paso “e” es un paso compuesto, por lo tanto, el caso anterior es más efectivo cuando se escribe de la siguiente manera:

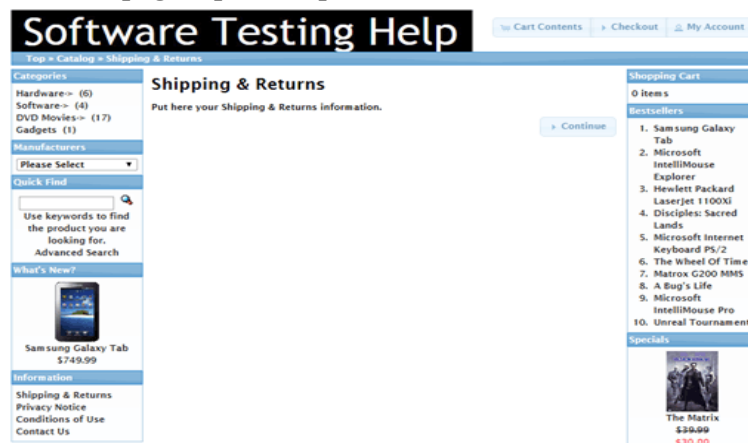
- a. Lanzar Amazon.com
- b. Busque un producto ingresando la palabra clave / nombre del producto en el campo 'Buscar' en la parte superior de la pantalla.
- c. De los resultados de búsqueda que se muestran, elija el primero.
- d. Haga clic en Agregar al carrito en la página de detalles del producto.
- e. *Haga clic en Pagar en la página del carrito de compras.*

- f. *Ingrese la información de CC, envío y facturación.*
- g. *Haga clic en Pagar.*
- h. Consulta la página de confirmación del pedido.

2. El comportamiento de la aplicación se toma como comportamiento esperado

Cada vez más proyectos tienen que afrontar esta situación en estos días. La falta de documentación, la programación extrema, los ciclos de desarrollo rápidos son algunas de las razones que nos obligan a confiar en la aplicación (una versión anterior más o menos) para escribir las pruebas o para basar las pruebas en sí.

Ejemplo: si la siguiente es la página para la que está escribiendo / diseñando los pasos de prueba:



Si los pasos de mi caso de prueba son los siguientes:

Caso 1:

1. Inicie el sitio de compras.
2. Haga clic en Envío y devolución. Resultado esperado: la página de envío y devolución se muestra con 'Ponga su información aquí' y un botón 'Continuar'.

Entonces, **esto es incorrecto**.

Caso 2:

1. Inicie el sitio de compras.
2. Haga clic en Envío y devolución.
3. En el cuadro de texto 'Ingrese el número de pedido' presente en esta pantalla, ingrese el número de pedido.
4. Haga clic en Continuar - Resultado esperado: se muestran los detalles del pedido relacionados con el envío y las devoluciones.

El caso 2 es un mejor caso de prueba porque a pesar de que la aplicación de referencia se comporta de manera incorrecta, solo la tomamos como una guía, **investigamos más y escribimos el comportamiento esperado según la funcionalidad correcta anticipada**.

3. Varias condiciones en un caso

Los siguientes son los pasos de prueba dentro de una prueba para una función de inicio de sesión.

Ejemplo:

- A. Ingrese detalles válidos y haga clic en Enviar.
- B. Deje el campo Nombre de usuario vacío. Haga clic en Enviar.
- C. Deje el campo de la contraseña vacío y haga clic en Enviar.
- D. Elija un nombre de usuario / contraseña que ya haya iniciado sesión y haga clic en Enviar.

Lo que tenían que ser 4 casos diferentes se combinan en uno. Podrías estar pensando: ¿Qué hay de malo en eso? Está ahorrando mucha documentación y lo que puedo hacer en 4, lo estoy haciendo en 1. Bueno, no del todo, ¿por qué?

- ¿Qué pasa si una de las condiciones falla? Tenemos que marcar toda la prueba como '**fallida**'. Si marcamos el caso completo como 'fallido', significa que las 4 condiciones no están funcionando, lo cual no es realmente cierto.
- Las pruebas deben tener un flujo. Desde la condición previa hasta el paso 1 y todos los pasos. Si sigo este caso, en el paso A, si tiene éxito, iniciaré sesión en la página, donde la opción 'iniciar sesión' ya no está disponible. Entonces, cuando llegue al paso B, ¿dónde ingresará el tester el nombre de usuario? Verás, el flujo está roto.

Por eso, escribir pruebas modulares, parece mucho trabajo, pero todo lo que necesitas es separar las cosas y usar a nuestros mejores amigos Ctrl + C y Ctrl + V para trabajar para nosotros.

Ejemplo:

Veamos cómo escribir casos de prueba de manera eficiente para una pantalla de 'Inicio de sesión' familiar y simple como se muestra en la siguiente figura. Los **enfoque de prueba** será casi el mismo incluso para pantallas complejas con más información y características críticas.



The image shows a login form on a light blue background. It has two input fields: 'User Name' with the text 'Administrator' and 'Password' with masked characters '*****'. Below the fields is a 'Login' button. At the bottom, there are two links: 'Forgot Password?' and 'Registration'.

1. El primer enfoque para cualquier proceso de caso de prueba será obtener un prototipo de pantalla (o marcos de alambre) como el anterior, si está disponible. Es posible que esto no esté disponible para algunas de las funcionalidades y depende de la importancia de diseñar un prototipo en las primeras etapas de desarrollo.

Pero, si un SRS (Especificación de requisitos de software) documento está disponible para el proyecto, la mayoría de los prototipos de pantalla son desarrollados por el equipo del proyecto. Este tipo de pantalla simplifica el trabajo del evaluador y aumenta la eficiencia de las pruebas.

2. A continuación, el **especificaciones de requisitos funcionales**. Depende del proceso de organización, estará disponible en un conjunto de múltiples documentos.

Por lo tanto, decida cuál es el mejor documento para redactar casos, ya sea un documento de requisitos del usuario o una especificación de requisitos funcionales (o incluso un documento SRS si el equipo de pruebas lo puede entender cómodamente) que proporcionará un flujo funcional completo de lo seleccionado. característica para ser probado.

3. Una vez que el prototipo de pantalla y las especificaciones funcionales están en su lugar, el evaluador debe comenzar a escribir los casos con el siguiente enfoque y criterio.

Pruebas de UI, Interfaz Usuaria: los controles / campos que son visibles para el usuario. Hay controles estáticos y dinámicos disponibles para probar la función. Por ejemplo, En la pantalla de inicio de sesión anterior, los textos de 'Nombre de usuario y contraseña' son campos estáticos que no requieren la interacción del usuario, solo para mostrar el texto.

Casos funcionales :Por otro lado, el botón Iniciar sesión y los Hipervínculos (¿Olvidó su contraseña? Y Registro) son campos dinámicos que requieren la interacción del usuario haciendo clic en los controles, que realizarán alguna acción después.

Casos de bases de datos :Una vez que el usuario ingresa el nombre de usuario y la contraseña, las pruebas se pueden escribir para verificar la base de datos relacionada, si el nombre de usuario y la contraseña están verificados en la base de datos y la tabla correctas y también el usuario tiene permiso para iniciar sesión en la aplicación bajo prueba.

Pruebas de proceso: esto está relacionado con el proceso (no las acciones asociadas con los controles visibles disponibles en la pantalla) asociado con la característica y la funcionalidad.

Por ejemplo, Si hace clic en el enlace “Olvidé mi contraseña” en la pantalla de muestra anterior, puede enviar un correo electrónico al usuario. Entonces, tal vez un correo electrónico deba probarse para el proceso y la confirmación adecuados.

4. Finalmente, mantenga **el Enfoque de BAOE (Básico, Alternativo, Opciones y Excepciones)** para la cobertura completa del flujo funcional y la característica a probar. Todos los conceptos deben aplicarse a pruebas positivas y negativas.

- i) Flujo **básico**
- ii) Flujo **alternativo**
- iii) **Opciones**
- iv) **Excepciones**

Por ejemplo, Veamos el enfoque **BAOE** simple para la pantalla de inicio de sesión

- I. **Flujo básico**: Ingrese la ruta URL del inicio de sesión en cualquier navegador e ingrese la información requerida e inicie sesión en la aplicación.
- II. **Flujo alternativo**: Instale la aplicación en un dispositivo móvil e ingrese la información requerida e inicie sesión en la aplicación.
- III. **Opciones**: ¿Cuáles son las opciones disponibles para llegar a la misma pantalla de inicio de sesión?

Ejemplo, después de iniciar sesión en la aplicación, al hacer clic en 'Cerrar sesión' puede aparecer la misma pantalla o, si el tiempo de espera de la sesión o la sesión expiró, el usuario puede acceder a la pantalla de inicio de sesión.

- IV. **Excepciones**: Cuáles son las excepciones si mis pruebas son negativas?

Ejemplo si se ingresan credenciales incorrectas en la pantalla de inicio de sesión, si el usuario recibirá un mensaje de error o ninguna acción asociada.

Con toda esta información en la mano, comencemos a escribir los caso de prueba para la pantalla de inicio de sesión, en un formato con la cobertura y trazabilidad completa y con información detallada. La secuencia lógica y la numeración de identificar el ' **ID de caso de prueba** ' será muy útil para un historial de ejecución de identificación rápida de casos de prueba.

Documento de caso de prueba

Revision History				
Sl.No.	Date	Version	Description	Author
1.	23/01/2015	1.0.0	Test Cases – Initial Draft	Tester A
2.	27/01/2015	1.0.1	Updated based on the reviewed and updated requirements specifications.	Tester A
3.	05/02/2015	1.0.2	Updated based on the peer review and traceability matrix.	Tester A
4.				
5.				

©www.SoftwareTestingHelp.com

Test Execution History					
Project ID	Project XX01	Project Name	Project A	Client Name	Client A
Test Manager	Mr.ABC	Reviewed By	Mr.XYZ	Reviewed On	10/02/2015

Test Case ID		Version Number	Iteration Cycle	Executed By	Execution Date	
From	To				From	To
XX01_Lgn_0001	XX01_Lgn_0022	1.0.1	1	Tester A	20/02/2015	21/02/2015
XX01_Lgn_0023	XX01_Lgn_0040	1.0.1	2	Tester A	21/02/2015	22/02/2015
XX01_Lgn_0041	XX01_Lgn_0053	1.0.1	2	Tester B	23/02/2015	23/02/2015
XX01_Lgn_0054	XX01_Lgn_0072	1.0.1	1	Tester A	28/02/2015	28/02/2015

Nota : Las columnas de prueba no se limitan al siguiente documento de prueba de muestra, que se puede mantener en una hoja de Excel para tener tantas columnas como sea necesario para una matriz de trazabilidad completa, a saber, prioridad, propósito de la prueba, tipo de prueba, ubicación de la captura de pantalla del error etc.,

Test Cases						
Test Case ID	Status	Severity	Steps to Reproduce	Expected Behaviour	Actual Behaviour	Result
XX01_Lgn_0001	Open	Blocker				Pass
XX01_Lgn_0002	Closed	Major				Pass
XX01_Lgn_0003	Reopened	Minor				Fail

Para facilitar la simplicidad y legibilidad de este documento, escribamos los pasos para reproducir, el comportamiento esperado y real de las pruebas para la pantalla de inicio de sesión en detalle a continuación.

Nota: se agregaría la columna **Comportamiento real** al final de esta plantilla si estuviera corriendo las pruebas

Ingeniería de Software : Generación de casos y datos de Prueba
Material obligatorio leer para el desarrollo de TP

No.	Pasos para reproducir	Comportamiento esperado
1.	Abra un navegador e ingrese la URL para la pantalla de inicio de sesión.	Debería mostrarse la pantalla de inicio de sesión.
2.	Instale la aplicación en el teléfono Android y ábrala.	Debería mostrarse la pantalla de inicio de sesión.
3.	Abra la pantalla de inicio de sesión y compruebe que los textos disponibles estén escritos correctamente.	El texto 'Nombre de usuario' y 'Contraseña' debe aparecer antes del cuadro de texto relacionado. El botón de inicio de sesión debe tener el título 'Iniciar sesión'. '¿Olvidó su contraseña?' Y 'Registro' deberían estar disponibles como enlaces.
4.	Introduzca el texto en el cuadro Nombre de usuario.	El texto se puede ingresar haciendo clic con el mouse o enfocarse usando la pestaña.
5.	Ingrese el texto en el cuadro Contraseña.	El texto se puede ingresar haciendo clic con el mouse o enfocarse usando la pestaña.
6.	Haga clic en ¿Olvidó su contraseña? Enlace.	Hacer clic en el enlace debería llevar al usuario a la pantalla relacionada.
7.	Haga clic en el enlace de registro	Hacer clic en el enlace debería llevar al usuario a la pantalla relacionada.
8.	Introduzca el nombre de usuario y la contraseña y haga clic en el botón Iniciar sesión.	Hacer clic en el botón Iniciar sesión debería llevar a la pantalla o aplicación relacionada.
9.	Vaya a la base de datos y verifique que el nombre correcto de la tabla esté validado con las credenciales de entrada.	El nombre de la tabla debe validarse y debe actualizarse un indicador de estado para un inicio de sesión exitoso o incorrecto.
10.	Haga clic en Iniciar sesión sin ingresar ningún texto en los cuadros Nombre de usuario y Contraseña.	Haga clic en el botón Iniciar sesión para alertar a un cuadro de mensaje 'El nombre de usuario y la contraseña son obligatorios'.
11.	Haga clic en Iniciar sesión sin ingresar texto en el cuadro Nombre de usuario, pero ingrese texto en el cuadro Contraseña.	Haga clic en el botón Iniciar sesión para alertar a un cuadro de mensaje 'La contraseña es obligatoria'.
12.	Haga clic en Iniciar sesión sin ingresar texto en el cuadro Contraseña, pero ingresando texto en el cuadro Nombre de usuario.	Haga clic en el botón Iniciar sesión para alertar a un cuadro de mensaje 'El nombre de usuario es obligatorio'.
13.	Ingrese el texto máximo permitido en los cuadros Nombre de usuario y Contraseña.	Debe aceptar el máximo permitido de 30 caracteres.
14.	Ingrese el nombre de usuario y la contraseña comenzando con los caracteres especiales.	No debe aceptar el texto que comience con caracteres especiales, lo cual no está permitido en el Registro.
15.	Ingrese el nombre de usuario y la contraseña comenzando con espacios en blanco.	No debe aceptar el texto indicando con espacios en blanco, lo cual no está permitido en Registro.
16.	Ingrese el texto en el campo de contraseña.	No debe mostrar el texto real en su lugar debe mostrar el símbolo de asterisco *.
17.	Actualiza la página de inicio de sesión.	La página debe actualizarse con los campos Nombre de usuario y Contraseña en blanco.

Ingeniería de Software : Generación de casos y datos de Prueba
Material obligatorio leer para el desarrollo de TP

18.	Ingrese el nombre de usuario.	Depende de la configuración de llenado automático del navegador, los nombres de usuario ingresados previamente deben mostrarse como un menú desplegable.
19.	Introduce la contraseña.	Depende de la configuración de autocompletar del navegador, las contraseñas ingresadas anteriormente NO deben mostrarse como un menú desplegable.
20.	Mueva el foco al enlace Olvidé mi contraseña usando Tab.	Deben poder utilizarse tanto el clic del ratón como la tecla Intro.
21.	Mueva el foco al enlace Registro usando Tab.	Deben poder utilizarse tanto el clic del ratón como la tecla Intro.
22.	Actualice la página de inicio de sesión y presione la tecla Intro.	El botón Iniciar sesión debe estar enfocado y la acción relacionada debe activarse.
23.	Actualice la página de inicio de sesión y presione la tecla Tab.	El primer foco en la pantalla de inicio de sesión debe ser el cuadro Nombre de usuario.
24.	Ingrese el usuario y la contraseña y deje la página de inicio de sesión inactiva durante 10 minutos.	La alerta del cuadro de mensaje 'Sesión caducada, ingrese nuevamente el nombre de usuario y contraseña' debe aparecer con los campos de nombre de usuario y contraseña limpios.
25.	Introduzca la URL de inicio de sesión en los navegadores Chrome, Firefox e Internet Explorer.	La misma pantalla de inicio de sesión debe mostrarse sin mucha desviación en el aspecto y la alineación del texto y los controles de formulario.
26.	Ingrese las credenciales de inicio de sesión y verifique la actividad de inicio de sesión en los navegadores Chrome, Firefox e Internet Explorer.	La acción del botón Iniciar sesión debe ser la misma en todos los navegadores.
27.	Verifique que el enlace Olvidé la contraseña y el registro no esté roto en los navegadores Chrome, Firefox e Internet Explorer.	Ambos enlaces deben llevar a las pantallas relativas en todos los navegadores.
28.	Verifique que la funcionalidad de inicio de sesión funcione correctamente en teléfonos móviles con Android.	La función de inicio de sesión debería funcionar de la misma manera que está disponible en la versión web.
29.	Verifique que la funcionalidad de inicio de sesión funcione correctamente en Tab y iPhones.	La función de inicio de sesión debería funcionar de la misma manera que está disponible en la versión web.
30.	Verificar la pantalla de inicio de sesión permite que los usuarios concurrentes del sistema y todos los usuarios obtengan la pantalla de inicio de sesión sin demoras y dentro del tiempo definido de 5-10 segundos.	Esto debe lograrse usando muchas combinaciones de sistema operativo y navegadores, ya sea física o virtualmente, o puede lograrse usando alguna herramienta de prueba de rendimiento / carga.

Recopilación de datos de prueba

Cuando se escribe el caso de prueba, la tarea más importante para cualquier tester es recopilar los datos de prueba. Muchos evaluadores omiten esta actividad y la pasan por alto con la suposición de que los casos de prueba se pueden ejecutar con algunos datos de muestra o datos ficticios y se pueden alimentar cuando los datos realmente se requieren. Este es un concepto erróneo crítico de alimentar datos de muestra o datos de entrada desde la memoria mental en el momento de ejecutar casos de prueba.

Si los datos no se recopilan y actualizan en el documento de prueba en el momento de escribir las pruebas, el evaluador pasaría anormalmente más tiempo para recopilar los datos en el momento de la ejecución de la prueba. Los datos de la prueba deben recopilarse tanto para casos positivos como negativos desde la perspectiva del flujo funcional de la característica.

Encuentre un documento de datos de prueba de muestra para las pruebas escritas anteriormente, que a su vez será útil sobre la eficacia con la que podemos recopilar los datos que facilitarán nuestro trabajo en el momento de la ejecución de la prueba.

Nro.	Propósito de los datos de prueba	Datos de prueba reales
1.	Pruebe el nombre de usuario y la contraseña adecuados	Administrador (admin2015)
2.	Pruebe la longitud máxima del nombre de usuario y la contraseña	Administrador del sistema principal (admin2015admin2015admin2015admin)
3.	Pruebe los espacios en blanco para el nombre de usuario y la contraseña	Ingrese espacios en blanco usando la tecla de espacio para el nombre de usuario y la contraseña
4.	Pruebe el nombre de usuario y la contraseña incorrectos	Admin (activado) (digx ## \$ taxk209)
5.	Pruebe el nombre de usuario y la contraseña con espacios no controlados entre ellos.	Admin istrador (admin 2015)
6.	Pruebe el nombre de usuario y la contraseña comenzando con caracteres especiales	\$% # @ # \$ Administrador (% # * # ** # admin)
7.	Pruebe el nombre de usuario y la contraseña con todos los caracteres pequeños	administrador (admin2015)
8.	Pruebe el nombre de usuario y la contraseña con todos los caracteres en mayúscula	ADMINISTRADOR (ADMIN2015)
9.	Pruebe el inicio de sesión con el mismo nombre de usuario y contraseña con varios sistemas simultáneamente al mismo tiempo.	Administrador (admin2015): para Chrome en la misma máquina y en una máquina diferente con el sistema operativo Windows XP, Windows 7, Windows 8 y Windows Server. Administrador (admin2015): para Firefox en la misma máquina y en una máquina diferente con el sistema operativo Windows XP, Windows 7, Windows 8 y Windows Server. Administrador (admin2015): para Internet Explorer en la misma máquina y en una máquina diferente con el sistema operativo Windows XP, Windows 7, Windows 8 y Windows Server.
10.	Pruebe el inicio de sesión con el nombre de usuario y la contraseña en la aplicación móvil.	Administrador (admin2015): para Safari y Opera en móviles, iPhones y tabletas con Android.

Qué es una prueba estándar en pruebas web?

Cree casos de prueba que estén completos: pasos, datos, variables, etc. Esto garantizará que los datos / variables no similares se puedan reemplazar simplemente cuando se requiera un caso de prueba similar.

- Los criterios de entrada y salida deben definirse adecuadamente.
- Los pasos modificables o la declaración en los pasos deben resaltarse en un color diferente para encontrarlos y reemplazarlos rápidamente.
- El lenguaje utilizado para la creación de casos de prueba estándar debe ser genérico.
- Todas las características de cada sitio web deben cubrirse en los casos de prueba.
- El nombre de los casos de prueba debe ser el nombre de la funcionalidad o la característica que cubre el caso de prueba. Esto facilitará mucho la búsqueda del caso de prueba del conjunto.
- Si hay una muestra básica o estándar, un archivo GUI o una captura de pantalla de la función, debe adjuntarse con los pasos correspondientes.

Al usar los consejos anteriores, uno puede crear un conjunto de scripts estándar y usarlos con modificaciones pequeñas o necesarias para diferentes sitios web.

El uso de un conjunto estándar de casos de prueba manuales para diferentes sitios web con modificaciones menores es la mejor manera de realizar una prueba de sitio web. Todo lo que necesitamos es crear y mantener los casos de prueba con los estándares y el uso adecuados.

Conclusión

- Mejorar la eficiencia de los casos de prueba no es un término simplemente definido, sino que es un ejercicio y se puede lograr a través de un proceso maduro y práctica regular.
- El equipo de pruebas no debe cansarse de involucrarse en la mejora de tales tareas, ya que es la mejor herramienta para lograr mayores logros en el mundo de la calidad, esto está probado en muchas de las organizaciones de prueba en todo el mundo en proyectos de misión crítica y aplicaciones complejas.

Mejoramiento y mantenimiento de los casos de prueba

Comprobación de los Casos de Prueba

En la prueba es fácil de probar, con precisión, lo que significa que, si el tester sigue las instrucciones, el resultado de aprobado o fallido será correcto. Se puede medir fácilmente por medio del tiempo que se tarda en ejecutar la prueba, y si el tester tiene que buscar o no aclaraciones en el proceso de prueba.

Lenguaje para mejorar la comprobación

Los pasos de los casos de prueba deben ser escritos en forma activa. El tester debe saber qué hacer, y cómo hacerlo.

- Por ejemplo, navegar en la página de la tienda online y preparar la lista de lo que va a comprar, para comparar los precios y la variedad con los datos disponibles.

Controlar longitud para mejorar la comprobación

Es necesario tener en cuenta la longitud de los casos de prueba para saber cuán compleja y precisa es la prueba.

- * **Un buen caso de prueba debe tener entre 8 y 16 pasos** —en el método paso a paso—, a menos que el tester Nro. 31 pueda guardar su trabajo.

- * Existen varias ventajas en mantener los casos de prueba cortos: se requiere menos tiempo y hay menos posibilidades de cometer errores, de necesitar ayuda o de alguna pérdida de datos.
- * Con base en la longitud de los casos de prueba es posible estimar con precisión el tiempo y el esfuerzo que se debe invertir en la prueba, lo mismo que sus resultados.
- * En los casos de prueba de matriz, una buena longitud oscila entre 18 y 20 minutos para la prueba.

Tipos de Mantenimiento

Si un problema es detectado por el usuario, inmediatamente puede notificarlo al administrador del sistema. Dicha petición debe ser atendida por el administrador y este procederá a diagnosticar de qué tipo de mantenimiento se trata, luego de finalizar las etapas de desarrollo de un sistema.

Atendiendo a estos fines, podemos establecer los siguientes **tipos de mantenimiento**:

- a) Correctivo.** Cambios precisos para corregir errores del producto de software.
- b) Evolutivo.** Incorporaciones, modificaciones y eliminaciones necesarias en un producto de software para cubrir la expansión o cambio en los requerimientos del usuario.
- c) Adaptativo.** Modificaciones que afectan los entornos en los que el sistema opera, por ejemplo, cambio en las configuraciones del hardware, software de base, gestores de base de datos, comunicaciones, etc.
- d) Perfectivo.** Acciones llevadas a cabo para mejorar la calidad interna de los sistemas en cualquiera de sus aspectos: restructuración de código, definición más clara del sistema y optimización del rendimiento y eficiencia.

Una vez identificado el tipo de mantenimiento y su origen se determina un tiempo razonable para su modificación y prueba, haciéndolo del conocimiento del usuario.

Si se trata de un mantenimiento correctivo, los cuales son más comunes, se verifica y reproduce el problema, o se estudia la viabilidad del cambio propuesto por el usuario. En ambos casos se identifican, según el tipo de mantenimiento de que se trate, cuál es la más adecuada. El plazo y urgencia de la solución a la petición se establece de acuerdo con el estudio anterior.

Las tareas de los procesos de desarrollo que va a ser necesario realizar son determinadas en función de los componentes del sistema actual afectados por la modificación. Estas tareas pertenecen a actividades de los procesos análisis, diseño, desarrollo y pruebas.

Por último, y antes de la aceptación del usuario, es preciso establecer un plan de pruebas de regresión que asegure la integración del sistema de información afectado.

Revisiones periódicas

El monitoreo permanente del sistema asegura que las necesidades de mantenimiento sean identificadas y satisfechas cuando resulte necesario. Cuando el sistema es de uso prolongado, se puede establecer un mecanismo para recibir retroalimentación de los usuarios como una forma efectiva para determinar las necesidades de mantenimiento y modificación.

A los sistemas se les debe dar mantenimiento para asegurar que continúen operando en el nivel mostrado durante la etapa de prueba. Si los sistemas se deterioran, existe el riesgo de que no se desempeñen conforme a los estándares requeridos.

Cambios en la lógica del sistema

Si un cambio se presentara en la lógica de programación del sistema, es posible realizar las modificaciones necesarias sin grandes complicaciones, ya que la parte de diseño y la lógica de programación se encuentran separadas, dando como resultado que la tarea de mantenimiento no se vuelva un gran problema. En este caso deben adaptarse o rehacerse los casos de prueba.

Definición, significado o concepto de información

Para tener claro el concepto de Información, tomaremos en cuenta diversos enfoques:

La información es parte fundamental y esencial de toda organización, ya sea pública o privada. Esta es la encargada de darle un nivel alto de competitividad frente a sus competidores, así como grandes posibilidades de desarrollo. Es por ello que esta información debe cumplir ciertas características tales como relevancia, exactitud, confiable y entre otras cualidades

La Información es un recurso vital, juega un papel muy importante dentro del largo periodo de vida de una organización. **Las empresas también usan otros recursos tales como materias primas, servicios y personas calificadas para los roles.** Pero hoy la información es el activo más importante de una empresa. La Información favorece a la resolución de problemas puesto que permite una adecuada toma de decisiones. Esta es la encargada de darle un nivel alto de competitividad frente a sus competidores y el mercado, así como grandes posibilidades de desarrollo

Diferencias entre Dato e Información:

Dato	Información
Representación Simbólica No tienen sentido semántico No transmiten mensaje Describen situaciones, hechos	Conjunto de datos procesados Conjunto de datos organizados Tienen un significado Transmiten un mensaje Permite la toma de decisiones Favorece a la resolución de problemas Incrementa el conocimiento
Un dato se define como una representación simbólica que por sí solo no transmite ningún mensaje y tampoco permiten tomar decisiones.	La información es parte fundamental y esencial de toda organización, ya sea pública o privada. Esta es la encargada de darle un nivel alto de competitividad frente a sus competidores, así como grandes posibilidades de desarrollo.
Ejemplo en una encuesta Nombre: Diego Tomás Apellidos: Torres Edad: 29 años Dirección: Av. Lima, Nro.340, PB	Continúa ejemplo de la encuesta: Si ahora obtenemos los datos de unos 140 encuestados de la misma provincia, partido y ciudad, sabiendo que son 33.415 habitantes en

Departamento/Zona/Partido: Quilmes Localidad: Bernal Provincia: Buenos Aires Teléfono: 01165432 Ocupación: técnico en redes Sueldo laboral: \$ 950.000	Bernal, los analizamos y procesamos se convierte en información. Ahora en base a ese conjunto de datos se puede concluir que el sueldo promedio es de \$650.00 y saber si el sueldo es suficiente para mantener la calidad de vida, entre otros.
Si se dan cuenta los datos presentados no transmiten mensaje y tampoco se puede tomar decisiones.	Al tener una relación y poder procesar los datos con un objetivo, pasa a ser información.

Comprensión

La información debe ser entendida para que sea utilizada, caso contrario no tendrá valor para la persona que recibe el mensaje.

Existen diversos factores que influyen en la comprensión de la información, las cuales se pasa a mencionar a continuación:

- a) **Preferencias del usuario.** Aquí se debe tener en cuenta que cada persona es diferente y por ende entiende de forma diferente, en tal caso algunas personas prefieren información en reportes numéricos, o con forma de gráficos o cuadros, otras prefieren una descripción narrativa, estadísticas, etc.
- b) **Conocimientos previos:** Para que la persona comprenda debe de tener algunos conocimientos adquiridos anteriormente; es decir, la persona asocia la memoria con el mensaje que se recibe.
- c) **Factores ambientales.** Para que la persona tenga una mayor comprensión va a depender del tiempo disponible que tenga, la confianza, las presiones de grupo, etc.
- d) **Lenguaje.** El mensaje debe ser transmitido en un lenguaje común, entendible en el negocio en que se está desarrollando la información y la toma de decisiones.

De la misma forma la información debe cumplir ciertas funciones dentro de la empresa, tanto gerenciales como operacionales

Sistemas de datos

Son sistemas que permiten la búsqueda y extracción de información. El sistema más usado es el de base de datos y estas permiten obtener elementos concretos de información a partir de una colección enorme de datos.

Una base de datos es una colección de datos relacionados.

Propiedades de una bases de datos

- * Representa algún aspecto del mundo real
- * La colección de datos es lógicamente coherente con un significado inherente
- * Su diseño y construcción tiene un propósito específico
- * Puede ser utilizada desde múltiples aplicaciones
- * Puede ser usada por diversos grupos de usuarios habilitados para consultar, modificar o dar de baja.
- * Está sujeta a un conjunto de restricciones

Las organizaciones crean varias copias de datos de la aplicación para utilizarlas en pruebas y desarrollo. A menudo, las organizaciones mantienen un fuerte control sobre los sistemas de producción, pero la seguridad de los datos no es tan fiable en los sistemas que no son de producción. Una organización debe tener conocimiento de los datos confidenciales en los sistemas de producción y asegurarse de que no aparezcan en el entorno de prueba. Los programadores no deben tener que reescribir los códigos para crear datos de prueba.

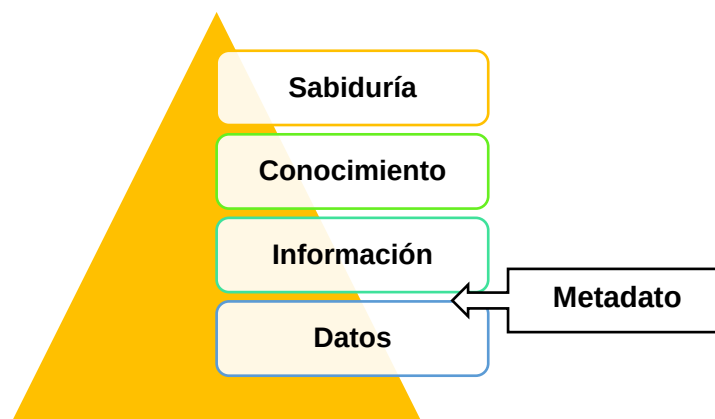
Hay herramientas que permiten crear una copia de menor tamaño de los datos de producción y enmascarar los datos confidenciales.

Una organización puede detectar las columnas sensibles en los datos de prueba y asegurarse de que las columnas sensibles están enmascaradas en ellos. Una organización también puede crear datos de prueba que no contengan datos confidenciales desde la base de datos de producción.

Las herramientas de generación de datos de prueba ayudan a los tester en carga, rendimiento, pruebas de estrés y también en pruebas de bases de datos. Los datos generados a través de estas herramientas también se pueden utilizar en otras bases de datos.

Algunas herramientas también brindan seguridad a la base de datos al reemplazar los datos confidenciales por uno ficticio. Al mismo tiempo, también conserva los datos confidenciales. Estas herramientas también brindan una opción para generar los datos generados en los scripts SQL. Por lo tanto, de esta manera, estas herramientas ayudan mucho en las pruebas y el desarrollo de aplicaciones.

Según la definición tradicional, **los metadatos son datos sobre datos**. En el sistema de depósito de datos, los metadatos pueden ayudar a los administradores del depósito de datos y a los desarrolladores del depósito o repositorio de datos a encontrar los datos que les interesan muy fácilmente. Los metadatos son datos que describen la estructura y los métodos de establecimiento de datos en el depósito de datos.



Datos de Prueba: Los datos de prueba (test data) son los datos que existen (por ejemplo, en una base de datos o en el caso de prueba) antes de que una prueba sea ejecutada, y que afecta o es afectado por el componente o sistema a probar

Obtención de datos : Proceso de detectar los metadatos de los sistemas de origen que incluyen contenido, como valores y frecuencias de datos, y la estructura como claves principales, claves externas y dependencias funcionales.

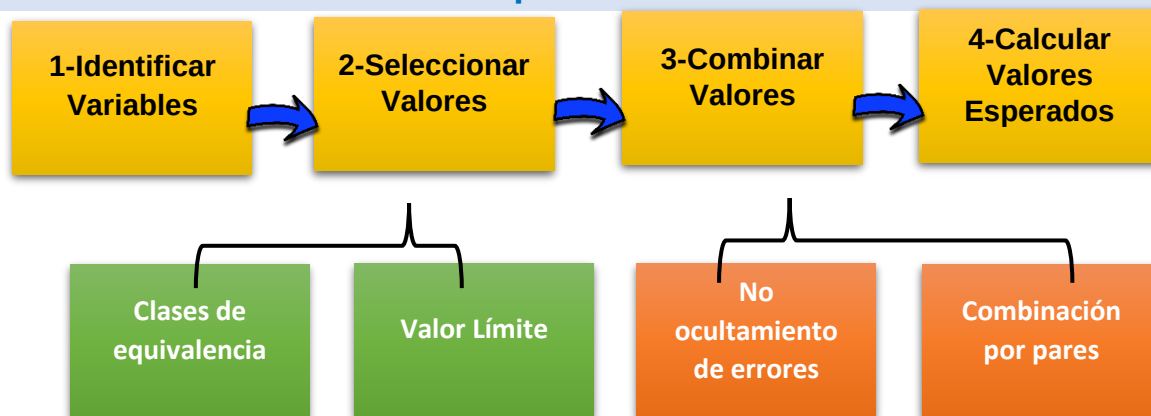
Generación de datos: El proceso para generar datos de prueba realistas para el entorno de prueba sin utilizar los datos de producción.

Enmascaramiento de datos: Proceso de sustitución de las columnas sensibles de los datos de origen con datos de prueba realistas.

Test Data Manager (TDM): Interfaz de usuario basada en web que se utiliza para configurar y ejecutar subconjuntos de datos, enmascaramiento de datos y operaciones de detección.

Administración de Datos de Prueba (TDM): Solución de informática que empaqueta operaciones de subconjuntos de datos, generación de datos y enmascaramiento de datos con el fin de proteger datos confidenciales y crear sistemas Lean ajenos a producción para prueba y desarrollo.

Proceso básico de diseño de pruebas



Este es un posible esquema de trabajo propuesto para diseñar datos de prueba para nuestra funcionalidad bajo prueba..

Primero se identifican las variables que están en juego en la funcionalidad luego, para cada variable se diseñan valores interesantes, para lo cual es útil aplicar técnicas de partición en clases de equivalencia y valores límites. Por último, estos valores deben combinarse de alguna forma, porque seguramente el **producto cartesiano** (todas las combinaciones posibles) nos daría una cantidad de casos de prueba muy grande, y quizá muchos de esos casos tendrán “poco valor agregado”.

1. Identificar Variables

El comportamiento esperado de cada funcionalidad a probar es afectado por una serie de variables que deben ser identificadas para el diseño de pruebas. Tal como indica el nombre, estamos hablando de variables, de cualquier entrada que al cambiar su valor hace que **cambie el comportamiento o resultado** del sistema bajo pruebas.

Qué puede conformar las variables de una funcionalidad:

 Entradas del usuario por pantalla.

- 🌀 Parámetros del sistema (en archivos de configuración, parámetros de invocación del programa, tablas de parámetros, etc.).
- 🌀 Estado de la base de datos (existencia –o no– de registros con determinados valores o propiedades).

Existen muchas entradas de un sistema que tal vez no afectan el comportamiento o resultado de lo que estamos ejecutando. Lo mismo para el estado inicial de la base de datos, tal vez existen cientos de registros, muchas tablas, etc., pero no todas afectan la operación que estamos probando. Se debe identificar qué variables son las que nos interesan, para focalizar el diseño de las pruebas y de los datos de prueba considerando estas variables, y no focalizar en otras pues esto nos hará diseñar y ejecutar más pruebas que quizá no aporten valor al conjunto de pruebas.

Veamos [un ejemplo simple](#), de una aplicación que permite ingresar facturas (*Invoices*), simplemente ingresando el cliente al que se le vende, fecha y líneas de producto, donde se indica cada producto y la cantidad de unidades que compra. Abajo se ve la pantalla web para el ingreso de facturas. El sistema verifica que los valores ingresados en los campos *Client Name* y *Product Name* existan en la base de datos. En el caso del producto, al ingresar un nombre, carga en la fila los valores para *Stock* y *Price*. Con el valor del precio, y al ingresar la cantidad de unidades deseada, se calcula el *Line Amount* de esa fila.

Line Id	Product Name	Stock	Price	Line Quantity	Line Amount
0		0	0.00	0	0.00
0		0	0.00	0	0.00
0		0	0.00	0	0.00
0		0	0.00	0	0.00
0		0	0.00	0	0.00

[New row]

Sub Total 0.00

Claramente el nombre de cliente es una variable, así como el producto y la cantidad del producto que se quiere facturar. Otras menos evidentes tal vez podrían ser:

- * Stock y precio del producto: a pesar de que son valores que no ingresa directamente el usuario, es interesante considerar qué pasa cuando se intenta comprar más cantidad de las que hay en stock, o si el precio es negativo o inválido.
 - Acá estamos considerando valores ya existentes en la base de datos, y los controlamos con la variable Producto, o sea, para variar estos valores necesitaremos ingresar productos con las características deseadas, y luego seleccionarlos.
- * Cantidad de líneas de la factura. Esta es una variable interesante generalmente en toda lista, la cantidad de elementos, donde vamos a querer ver qué pasa cuando intentamos crear una factura sin ninguna línea de producto, o con una cantidad muy grande.
- * Subtotal de la línea. Hay variables que son cálculos de otras, como en este caso que

se trata de la multiplicación de la cantidad por el precio del producto. Sería interesante ver qué pasa con valores muy grandes, verificando que no se produzca un *overflow* en la operación.

Analizando más la especificación del sistema, nos encontramos con que se registra el balance del cliente, con lo cual, si este tiene un balance negativo, no se le habilitará el pago a crédito. Por esto es interesante considerar la variable *balance* asociada al cliente ingresado. Esto también corresponde a un valor de la base de datos.

2. Seleccionar Valores Interesantes

Una vez que se identifican las variables en juego en la funcionalidad bajo pruebas, **para cada una hay que seleccionar los valores que se le asignarán en las pruebas**. Para cada variable se decidirá utilizar distintos datos, los cuales sean interesantes desde el punto de vista del testing.

Para diseñar los conjuntos de datos de prueba se analizan las reglas del negocio, los tipos de datos, y los cálculos a realizar, para poder determinar primero que nada las **clases de equivalencia**. En términos de testing, se considera que un grupo de valores pertenece a una misma clase de equivalencias si deben producir un comportamiento similar en el sistema.

Considerando eso, podríamos decir que al elegir un representante cualquiera de cada clase de equivalencia será “suficiente” para probar el total de las entradas.

¿Cómo se identifican las clases de equivalencia? Pues pensando en opciones “significativamente diferentes”, o sea, que generan distintos comportamientos.

Ejemplo: si tenemos una variable para ingresar un identificador que acepta cadenas de entre 5 y 10 caracteres podremos pensar en “Pau”, que es muy corto por lo que debe dar un error, “Paulina” que es una entrada válida, “Paulina Pilar García” que es muy largo, por lo que debe dar error. En cambio, “Paulina” y “Gonzalo” son ambas válidas, no son “significativamente diferentes”, por lo que no corresponden a la misma clase de equivalencia, ambas deberían generar el mismo comportamiento en el sistema.

Podemos distinguir opciones significativamente diferentes cuando:

- ✚ Disparan distinto flujo sobre el sistema (al ingresar un valor inválido me lleva a otra pantalla distinta).
- ✚ Dispara un mensaje de error diferente (no es lo mismo ingresar una contraseña o password corta que uno vacía).
- ✚ Cambia la apariencia de la interfaz del usuario, o el contenido del formulario (al seleccionar pago con tarjeta de crédito aparecen los campos para ingresar el número de tarjeta, titular y fecha de expiración, pero si se selecciona pago por transferencia entonces aparecen los campos para ingresar información de la cuenta bancaria).

- ‡ Valores por defecto o cambiados (en ocasiones el sistema sugiere datos que se habían ingresado previamente para un cliente, como por ejemplo su dirección, pero si se cambian en el formulario de entrada entonces el sistema actualizará ese dato).
- ‡ Cambios de comportamiento por distintas configuraciones o distintos países (formato de números, formatos de fecha, moneda, etc.).

Para seleccionar los representantes de cada clase, **lo primero es definir cuáles son clases válidas e inválidas.**

Ejemplo, si estamos hablando de un campo de entrada que es para indicar la edad de una persona, y se sabe que el comportamiento cambia respecto a si se trata de un menor de edad que si se trata de un mayor de 18 años, entonces las clases que se pueden definir son:

- ★ Clase 1 = (-infinito, 0)
- ★ Clase 2 = (0, 18)
- ★ Clase 3 = (18, 100)
- ★ Clase 4 = (100, infinito)
- ★ Clase 5 = (a, b, c, %, #, /, i, *, \$,)

Sabemos que las clases 1 y 4 son inválidas porque no habrá edades mayores de 100 (normalmente) y tampoco son válidos los números negativos. Se podría definir una quinta clase, también inválida, que esté compuesta por cualquier carácter no numérico.

Luego, las clases 2 y 3 son válidas, y será interesante seleccionar representantes de cada una, como podría ser el valor 15 para la clase 2 y el valor 45 para la clase 3. Se considera "0" para los bebés entre 0 y 364 días de nacido.

Además, serán también interesantes los valores límites de las clases de equivalencia, es decir, para la clase 2 el valor 0, el 17 y 18. Probar con estos valores tiene sentido ya que un error muy común en los programadores es el de utilizar una comparación de mayor en lugar de mayor o igual, o similares. Este tipo de errores se pueden verificar solo con los **valores que están al borde de la condición.**

Observemos aquí que comienza a tener más sentido la **separación entre caso de prueba abstracto y caso de prueba específico**. El caso de prueba abstracto podría estar definido haciendo referencia a qué clase de equivalencia se utiliza en la prueba, y luego se instancia con un representante de esa clase de equivalencia.

Caso de prueba abstracto o de alto nivel es un caso de prueba sin valores concretos (nivel de implementación) para datos de entrada y resultados esperados. Se utilizan operadores lógicos, las instancias de los valores reales no se han definido aún y/o no están disponibles.

Caso de prueba específica o de bajo nivel es un caso de prueba con valores específicos, que comunica las condiciones específicas que deben validarse para habilitar una valoración de algunos aspectos concretos de los elementos del destino de la prueba.

Un caso de prueba difiere de una idea de prueba, en que el caso de prueba es una especificación formada más completamente de la prueba. Los casos de prueba pueden estar motivados por muchas cosas, pero habitualmente incluirán un subconjunto de requisitos, características de rendimiento y los riesgos que afectan al proyecto.

Ingeniería de Software : Generación de casos y datos de Prueba
Material obligatorio leer para el desarrollo de TP

Como norma general, las **especificaciones de caso de prueba** son más útiles donde la implementación de prueba será demasiado compleja para comprenderse por sí misma sin el soporte de una explicación más abstracta proporcionada por el caso de prueba.

EJEMPLO: funcionalidad de dar de alta un cliente se realiza con la pantalla que muestra:

Client

Id 0

First Name

Last Name

Age

Country Name

City Name

Address

Balance

El identificador "Id" es autogenerado al confirmar la creación. Los campos "First name" y "Last name" (nombre y apellido) se guardan en campos del tipo "Char(30)" (Carácter) en la base de datos. El campo "Address" (dirección) está definido como "Char(100)". Tanto "Country Name" (país) como "City Name" (ciudad) se presentan en *combo-boxes* cargados con los valores válidos en la base de datos. Los clientes son tratados distinto según si es del mismo país o si es extranjero (por impuestos que se deben aplicar). Solo se pueden dar de alta clientes mayores de edad.

Siguiendo con el ejemplo, las variables con sus clases válidas e inválidas y sus valores interesantes, se podrían diseñar como muestra la Tabla

Variable	Clases de equivalencia	Válida o Inválida	Valores interesantes
Nombre	Hasta 30 caracteres	válida	"Paulina Pía"
	Más de 30	Inválida	String de 31 caracteres
	Vacía	Inválida	""
Apellido	Hasta 30 caracteres	Válida	"García"
	Más de 30	Inválida	String de 31 caracteres
	Vacía	Inválida	""
Edad	0<=x<=18 x>=18	Válida	18, 26, 30, 62, 100
	Negativos	Inválida	-1, -17, -18
	Caracteres no numéricos	Inválida	a, B, c, \$, &, +, fg, #
País	Cualquiera extranjero	Válida	Perú, Uruguay, Brasil, España, Rusia, etc.
	Local	Válida	Argentina
	Vacío	Inválida	""
Provincia	Cualquiera (según el país elegido)	Válida	Buenos Aires, Córdoba, Tucumán, Mendoza, San Luis, Salta, etc.
	Vacía	Inválida	""
Departamento, Partido o Regiones	Cualquiera (según la provincia elegida)	Válida (para Mza.)	Mendoza, Godoy Cruz, Luján, Maipu, Guaymallén, Las Heras, San Rafael, etc.
Ciudad	Cualquiera (según datos anteriores)	Válida (para Mendoza)	Quinta Sección, San José, Dorrego, Coquimbito, Potrerillos, Uspallata, Pedriel, Villa Hipódromo, 25 de Mayo,,etc.
	Vacía	Inválida	""
Dirección	Hasta 100 caracteres	Válida	"Lavalle, 1080"
	Más de 100	Inválida	String de 101 caracteres
	Vacía	Inválida	""

Análisis de Clases de equivalencia

Excepto para las aplicaciones de software más triviales, por lo general, se considera imposible probar todas las combinaciones de entradas factibles lógicamente para un sistema de software. Por consiguiente, seleccionar un subconjunto óptimo que ofrezca la máxima probabilidad de encontrar el mayor número de errores es una tarea importante y que vale la pena que lleven a cabo los testers.

Clases de equivalencia: clasificación de valores equivalentes para los que un objeto se espera que se comporte de forma similar. Esta técnica se puede aplicar para ayudar en el análisis de las pruebas más significativas que deben llevarse a cabo cuando hay demasiadas pruebas potenciales que se pueden ejecutar en el tiempo disponible.

Realizar pruebas basadas en el análisis de clase de equivalencia es una forma de análisis de prueba de caja negra (prueba sin meterse en el código) que intenta reducir el número total de pruebas potenciales a un conjunto mínimo de pruebas que revelan tantos errores como sea posible.

Este método particiona el conjunto de entradas y salidas en un número finito de clases de equivalencia que permite seleccionar un valor de prueba representativo para cada clase. La prueba resultado del valor representativo para una clase es el "equivalente" a los otros valores de la misma clase. Si no se encuentra ningún error en la prueba del valor representativo, se estima que todos los demás valores "equivalentes" tampoco hubieran identificado ningún error.

El poder de las clases de equivalencia yace en su capacidad de orientar al tester mediante la estrategia de pruebas para reducir la explosión combinatoria de las pruebas potencialmente necesarias. La técnica proporciona unas bases lógicas por la cuales un subconjunto del número total concebible de pruebas se puede seleccionar.

Aquí se muestran algunas categorías de áreas de problemas para grandes números de pruebas que se pueden beneficiar de la consideración de clases de equivalencia:

1. Combinaciones de variables independientes
2. Variables dependientes basadas en relaciones jerárquicas
3. Variables dependientes basadas en relaciones temporales
4. Relaciones en clúster basadas en ejemplares del mercado
5. Relaciones complejas que se pueden modelar

Estrategias

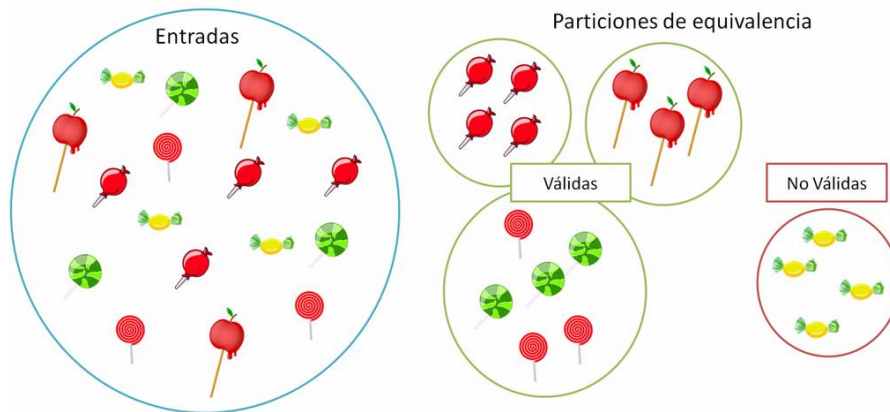
Existen técnicas y estrategias diferentes que se pueden utilizar en la prueba de partición de equivalencia. Estos son algunos ejemplos:

1. Partición de clase de equivalencia

Intenta reducir el número total de casos de prueba necesarios al particionar las condiciones de entrada en un número finito de clases de equivalencia.

Se han clasificado ambos tipos de clases de equivalencia: el conjunto de entradas válidas en el programa se considera como la *clase de equivalencia válida*, y todas las demás entradas se incluyen en la *clase de equivalencia no válida*.

Ejemplo:



Cada clase se llama partición de equivalencia porque todos elementos en la clase prueban la misma cosa. Si uno de los elementos de una clase provoca un error, todos los de esa clase probablemente lo harán. La idea principal de esta técnica es **identificar los casos de prueba usando un elemento de cada clase de equivalencia**.

Existen dos clases de equivalencia:

- ★ **Clases válidas:** valores de entrada válidos. No quiere decir que tu programa no vaya a fallar con estas entradas, sino que no debería fallar.
- ★ **Clases no válidas:** valores de entrada no válidos. Representan los valores de entrada que son erróneos y a los que el programa debería responder o bien rechazándolos o manejando una excepción.

Establecer las particiones de equivalencia puede ser un proceso subjetivo, por lo que dos personas podrían obtener clases de equivalencia distintas. La clave está en elegir las relaciones que determinan esas clases.

Las clases de equivalencia se definen según una serie de directrices:

- ◇ **Rangos:** si una entrada está condicionada a un rango de valores (por ejemplo, edades comprendidas entre 18 y 65 años ambos inclusive), se define una clase de equivalencia válida para todos los valores pertenecientes al rango y dos no válidas, una para los valores menores al límite inferior del rango y otra para los mayores al límite superior.
- ◇ **Valor específico:** análoga a la anterior, solo que el rango quedaría restringido a un único valor (la edad es de 18 años). Se siguen creando igualmente 1 clase de equivalencia válida y dos no válidas.
- ◇ **Número de valores:** también como las anteriores, pero teniendo en cuenta el número de valores introducidos (1 a 5 pasajeros sería la clase válida, 0 pasajeros y más de 5 pasajeros las no válidas).
- ◇ **Perteneciente a un conjunto:** cuando un valor es válido si pertenece a un conjunto (días de la semana en minúscula: «lunes», «martes»...) se define una clase de equivalencia válida para cada uno de los valores del conjunto, además de una no válida, para aquellos valores que no pertenecen a él.

- ◇ **Valor lógico:** se corresponde a la evaluación de una condición (por ejemplo, la edad debe ser mayor o igual a 18). Se establece una clase de equivalencia válida (se cumple la condición) y otra no válida (no se cumple).

Ejemplo: Días de la semana en formato numérico

La condición de entrada reflejaría que sólo se podrían introducir números del 1 al 7, ambos inclusive. Identificaríamos 2 clases:

- Clase válida: $1 \leq \text{día} \leq 7$
- 2 clases no válidas: una cuando $\text{día} < 1$ y otra para $\text{día} > 7$

Ejemplo: Colores RGB en formato String minúscula

El valor de entrada sólo puede corresponder a uno de los colores RGB (Led Rojo, Azul y verde) escrito en minúscula: «red», «green», «blue». Se supone que cada una de esas entradas se debería manejar de formas distintas en el programa.

- 3 clases de equivalencia válidas, una para cada uno de los valores de entrada «red», «green» y «blue».
- Una clase inválida que incluiría aquellos colores no especificados en la condición.

Ejemplo: nombre de países y ciudades con primera letra con mayúscula

- Clase válida para cadenas cuya primera letra es una mayúscula (se cumple la condición)
- Clase no válida para valores que no cumplen la condición: cadenas que comienzan en minúscula.

Conjunto de directrices para identificar clases de equivalencia

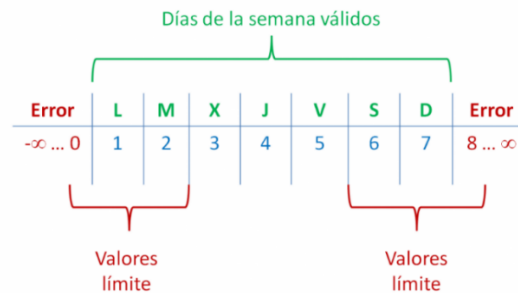
- * Si una condición de entrada especifica un rango valores (por ejemplo, el programa "acepta valores de 10 a 100"), se identifican una clase de equivalencia válida (de 10 a 100) y dos clases de equivalencia no válidas (menor que 10 y mayor que 100).
- * Si una condición de entrada especifica un conjunto de valores (por ejemplo, "la tela ser de varios colores: ROJO, BLANCO, NEGRO, VERDE, AZUL "), se identifica una clase de equivalencia válida (los valores válidos) y una clase de equivalencia no válida (todos los demás valores no válidos). Cada valor de la clase de equivalencia válida se debe manejar de modo diferente.
- * Si la condición de entrada se especifica como una situación de obligación (por ejemplo, "la cadena de caracteres de entrada debe ser en mayúsculas"), se identifica una clase de equivalencia válida (caracteres en mayúsculas: ALBA, NUBE, SOL) y una clase de equivalencia no válida (todas las demás entradas excepto los caracteres en mayúsculas: bala, vela, plato, etc.).
- * Todo lo que finaliza "mucho" antes de que se lleve a cabo la tarea es una clase de equivalencia. Todo lo que se realiza durante un intervalo de tiempo corto antes de que finalice el programa es otra clase. Todo lo que se realiza antes de que el programa inicie otra operación es otra clase.
- * La partición del suceso de salida se basa en las entradas del programa. Aunque clases de equivalencia de entrada diferentes pueden tener el mismo tipo de suceso de salida, las clases de equivalencia de entrada se deben seguir tratando de forma distinta.

2. Análisis de valor de límite

En cada una de las clases de equivalencia, se considera que las condiciones de límite tienen un mayor índice de éxito en la identificación de las anomalías resultantes que las condiciones que no son de límite. **Las condiciones de límite son los valores que se encuentran en, inmediatamente por encima o por debajo del límite o los "bordes" de cada clase de equivalencia.**

Las pruebas que son resultado de las condiciones de límite utilizan valores que se encuentran en el mínimo (min), justo encima del mínimo (min+), justo debajo del máximo (max-) y el máximo (max) del rango que se debe probar.

Ejemplo: si el dato de entrada corresponde a un número que nos indica el día de la semana en el que estamos, deberíamos definir el rango de valores válidos como: $1 \leq \text{día} \leq 7$. En ese caso, los casos de prueba resultantes a aplicar serían 0, 1, 2, 6, 7 y 8.



Al probar los valores de límite, los testers eligen algunos casos de prueba para cada clase de equivalencia. Para la muestra de pruebas relativamente pequeña, la probabilidad de descubrimiento de anomalías es alta. Se ha reducido al tester parte de la carga que supone probar una amplia población de casos en una clase de valores equivalente que es improbable que produzca diferencias importantes en los resultados de la prueba.

Algunas recomendaciones al elegir valores de límite:

- a) Para un entero, si el rango de entrada válido es de 10 a 100, probar 9, 10, 100, 101.
- b) Si un programa espera una letra en mayúsculas, probar de la A a la Z de límite. Probar también @ y [puesto que, en el código ASCII, @ está justo debajo de la A y [está justo después de la Z. Aquí se considera el diseño del teclado.
- c) Si la entrada o la salida de un programa es un conjunto ordenado, conviene prestar atención al primer y al último elemento del conjunto.
- d) Si la suma de las entradas debe ser un número específico (n), probar el programa donde la suma sea n-1, n, o bien, n+1.
- e) Si el programa acepta una lista, probar los valores de la lista. Todos los demás valores no son válidos, probar valores no existentes. Por ejemplo, si la lista son días del mes, tratar de ingresar 32 o 33. Y si considera meses en particular, probar de poner por ejemplo 31 de noviembre o 31 de marzo, que no existe.
- f) Al leer o escribir un archivo, comprobar los caracteres primero y último del archivo.
- g) La denominación de moneda más pequeña es un céntimo o equivalente. Si el programa acepta un rango específico, de la a a la b, probar a -0.01 y b +0.01.
- h) Para una variable con varios rangos, cada rango es una clase de equivalencia. Si los sub-rangos no están solapados, probar los valores de los límites, después del límite superior y debajo del límite inferior.

Valores límite en entradas con varios valores: cuando una entrada indica que se deben introducir varios valores, se deben diseñar casos de prueba considerando el **número de valores** como un **rango de valores** de forma que apliquemos a ese rango los valores límite para que el usuario tenga que introducir $n-1$, n , $n+1$, $m-1$, m y $m+1$ valores.

Por ejemplo, imagina un programa que recoge un mínimo de 3 notas y un máximo de 9 para elaborar una nota media de prácticas realizadas. Deberíamos elaborar casos de prueba en los que el usuario deba introducir 2, 3, 4, 8, 9 y 10 notas.

Valores límite en los datos de salida: si te es posible también debes aplicar este tipo de pruebas a los datos de salida. Imagina que, en el ejemplo último, la nota media que va a obtenerse como salida debe estar entre 0 y 10, deberías usar el análisis de valores límite con el fin de crear casos de prueba que generen los siguientes valores de notas medias: -1, 0, 1, 9, 10 y 11.

Por otro lado, si tienes un programa que estudiando los datos de compra de los usuarios de un supermercado obtenga una lista con los 5 productos más comprados. En ese caso deberíamos tratar de generar casos de prueba para que el programa genere una lista con 4 elementos, otra con 5 y otra con 6.

Ten en cuenta que no siempre podrás conseguir entradas que te permitan generar resultados fuera del rango de salidas. En los casos anteriores, a no ser que el programa esté mal diseñado y/o codificado difícilmente conseguirás medias de -1 o de 11 (debería permitirte introducir notas negativas o notas mayores de 10) o una lista con los 6 productos más vendidos. Sin embargo, sí que podrías obtener una lista con 4 productos si en los datos de ventas sólo se incluyen 4 productos.

3. Valores especiales

Después de intentar las dos estrategias de análisis de límite anteriores, un tester con experiencia puede observar las entradas de programa para descubrir todos los casos de "valores especiales" que, una vez más son, potencialmente, fuentes valiosas para revelar anomalías de software.

Algunos ejemplos:

1. Para un tipo de dato entero, se debe probar siempre cero si se encuentra en la clase de equivalencia válida.
2. Al probar la hora (hora, minuto y segundo), se debe probar siempre 59 y 0 como el límite superior e inferior para cada campo, sin tener en cuenta la restricción que tenga la variable de entrada. Así, excepto los valores de límite de la entrada, -1, 0, 59 y 60 siempre deben ser casos de prueba.
3. Al probar la fecha (año, mes y día), se deben incluir numerosos casos de prueba como, por ejemplo, un número de días en un mes específico, el número de días de febrero en año bisiesto o el número de días en años no bisiestos.

Cada conjunto de datos de prueba debe considerar varios aspectos, que incluyen los siguientes:

- * **Las condiciones previas necesarias de la configuración de entorno de prueba** que se supone que existen inmediatamente antes de que se consuman los datos de prueba.
- * **Las características únicas de los datos de prueba.** Estos datos pueden tener distintos formatos: desde valores de texto alfanuméricos estándar a datos sensoriales como, por

ejemplo, información auditiva o visual. Los datos de prueba pueden especificarse como un rango válido, en lugar de como un único valor, que debe utilizarse durante una prueba.

- * **Las dependencias entre los elementos de datos de prueba.**
- * **Una explicación descriptiva de la condición que se está probando**, a menudo definida en términos de cuál es la anomalía si se encuentra que la condición que se está probando es falsa.

Cuando se gestionan aparte de los aspectos de procedimiento de la prueba, los datos de prueba habilitan las características únicas de la prueba para que se modifiquen independientemente.

Tanto el contenido como el formato de los datos de prueba pueden requerir modificaciones para cubrir las necesidades de cada organización y proyecto concretos.

Cuando los datos de prueba se gestionan independientemente de las cuestiones de procedimiento de la prueba, se utilizan varios estilos distintos de almacenamiento:

- Un formato simple de archivo de texto ASCII, delimitado por caracteres especiales o columnas de ancho fijo.
- Un formato básico de hoja de cálculo o sistema de base de datos, como por ejemplo Microsoft® Excel® o Microsoft® Access®.
- Alguna forma de cálculo generado por programa de los datos de prueba.
- Alguna forma de captura, extracción o conversión de los datos de prueba a partir de una fuente original.
- Un sistema complejo de gestión de bases de datos relacionales (RDBMS) o de objetos (ODBMS). Muchos equipos de prueba utilizan la misma base de datos para gestionar los datos de prueba que la utilizada por el software que se está desarrollando. Esto a menudo resulta ventajoso ya que permite un acceso rápido a administradores y diseñadores de bases de datos cualificados que pueden proporcionar consejos y soporte al equipo de prueba.

Es habitual que varios elementos de datos de prueba se especifiquen en un único contenedor de almacenamiento, normalmente agrupados según la finalidad u objetivo general de las pruebas.

3. Combinación de Valores

Cuando diseñamos casos de prueba se utilizan datos para sus entradas y salidas esperadas, y por lo tanto también diseñamos datos de prueba para cada variable que está en juego. Cada caso de prueba se ejecutará con distintos juegos de datos, pero ¿es necesario utilizar todas las combinaciones posibles? Con casos muy simples, que manejan pocas variables y pocos valores, ya podemos observar que la cantidad de ejecuciones que necesitamos serían muchas, por lo que tenemos que intentar encontrar las combinaciones de datos que tienen más valor para el testing, o sea, las que tienen más probabilidad de encontrar errores.

Las **técnicas de testing combinatorio** permiten seleccionar una cantidad acotada de datos de prueba, siguiendo teorías de errores que indican cómo combinar los datos para aumentar esa probabilidad de encontrar errores, es decir, se busca obtener el conjunto de datos más eficiente posible (eficiente = encontrar la mayor cantidad de errores con la menor cantidad de ejecuciones).

Combinación por pares

Una de las técnicas de combinación de datos más usada es all-pairs, o sea todos los pares, y justamente lo que se hace es intentar utilizar todos los pares posibles.

Ejemplo: imaginen que tenemos 3 variables para un caso de prueba sencillo, y se seleccionaron valores interesantes para cada variable como muestra el siguiente ejemplo:

Funcionalidad bajo pruebas: Solicitud de servicio por parte de un cliente.

Variables en juego, y conjuntos de valores interesantes:

- Canal: {Presencial, Telefónico, e-mail}
- Prioridad: {Urgente, Alta, Media, Baja}
- Tipo de Servicio: {Adhesión Tarjeta Débito, Adhesión Tarjeta Crédito, Adhesión e-commerce}

El caso de prueba (Presencial, Media, Adhesión Tarjeta Débito) cubre los pares (Presencial, Media), (Presencial, Adhesión Tarjeta Débito) y (Media, Adhesión Tarjeta Débito).

Tendríamos que diseñar pruebas para cubrir todos los pares posibles. Queda claro que intentar hacer esto manualmente sería una tarea muy costosa.

Existen herramientas que aplican distintas estrategias para conseguir combinaciones de datos que cubran todos los pares (e incluso no solo pares, sino tripletas, o combinaciones de “n” elementos). Entre ellas vamos a mostrar aquí como ejemplo: CTweb (Combinatorial Testing Web ctweb.abstracta.com.uy), la cual implementa diferentes algoritmos para conseguir cubrir todos los pares de distintos conjuntos de datos de prueba (además de otras funcionalidades muy útiles).

Dentro de los distintos algoritmos existentes, el que nos resulta generalmente más útil es el llamado **PROW** (*pairwise with restrictions, order and weight=por pares con restricciones, orden y peso*)

Utilizando este algoritmo podemos excluir pares que nos resulten inválidos, e incluso darles distintas prioridades a los distintos pares, pues siempre será necesario repetir algunos pares para lograr todas las

combinaciones, entonces quizá sea más interesante que se repita un par que otro, ya que esa combinación de datos ocurre más en la realidad que el resto.

Sigamos el ejemplo con la herramienta. Primero ingresamos al sistema y definimos las variables con sus valores interesantes, seleccionamos el algoritmo PROW y presionamos el botón “Execute”

Ingeniería de Software : Generación de casos y datos de Prueba

Material obligatorio leer para el desarrollo de TP

Algorithms	Data																				
☐ All combinations (exponential cost) ☐ Each choice (very low cost) ☐ Antirandom (exponential cost) ☐ Comb (lineal cost) ☐ Genetic ☐ Costly pairwise (exponential cost) ☐ AETG (polynomial cost) ☒ PROW (polynomial cost) ☐ Customizable pairwise (exponential cost) ☐ Bacteriologic ☐ Random (lineal cost) Execute Verbose: ☐	Add set Add row <table border="1"> <thead> <tr> <th></th> <th>Canal</th> <th>Prioridad</th> <th>Tipo de Servicio</th> </tr> </thead> <tbody> <tr> <td>0</td> <td>Presencial</td> <td>Urgente</td> <td>Adhesión Tarjeta Débito</td> </tr> <tr> <td>1</td> <td>Telefónico</td> <td>Alta</td> <td>Adhesión Tarjeta Crédito</td> </tr> <tr> <td>2</td> <td>e-mail</td> <td>Media</td> <td>Adhesión e-commerce</td> </tr> <tr> <td>3</td> <td></td> <td>Baja</td> <td></td> </tr> </tbody> </table> Expression to generate test cases: Example <pre> public void testTCNUMBER { ClassUnderTest o=new ClassUnderTest(#A, #B); o.method1(#C); double result=o.method2(#C, #B, #A); ORACLE } </pre> Instructions With the <i>PROW</i> algorithm (Fairwise with Restrictions, Order and Weight), you must: 1. In a first step, execute the algorithm (<i>Execute</i> button) to see the pair tables. 2. Secondly, check those pairs to be removed. 3. Thirdly, assign a selection factor to those pairs you are interested in using in more test cases. 4. To effectively get the test cases, press again the <i>Execute</i> button		Canal	Prioridad	Tipo de Servicio	0	Presencial	Urgente	Adhesión Tarjeta Débito	1	Telefónico	Alta	Adhesión Tarjeta Crédito	2	e-mail	Media	Adhesión e-commerce	3		Baja	
	Canal	Prioridad	Tipo de Servicio																		
0	Presencial	Urgente	Adhesión Tarjeta Débito																		
1	Telefónico	Alta	Adhesión Tarjeta Crédito																		
2	e-mail	Media	Adhesión e-commerce																		
3		Baja																			

Algorithm "prow"

Check below the pairs to be removed and assign weights to pairs

12 pairs in (A, B)			9 pairs in (A, C)			12 pairs in (B, C)		
Elements	Remove	Sel. factor	Elements	Remove	Sel. factor	Elements	Remove	Sel. factor
(Presencial, Urgente)	☐	3.0	(Presencial, Adhesión Tarjeta Débito)	☐	2.0	(Urgente, Adhesión Tarjeta Débito)	☐	3.0
(Presencial, Alta)	☐	2.0	(Presencial, Adhesión Tarjeta Crédito)	☐	2.0	(Urgente, Adhesión Tarjeta Crédito)	☐	3.0
(Presencial, Media)	☐	1.0	(Presencial, Adhesión e-commerce)	☐	0.0	(Urgente, Adhesión e-commerce)	☐	0.0
(Presencial, Baja)	☐	1.0	(Telefónico, Adhesión Tarjeta Débito)	☐	1.0	(Alta, Adhesión Tarjeta Débito)	☐	2.0
(Telefónico, Urgente)	☐	0.5	(Telefónico, Adhesión Tarjeta Crédito)	☐	1.0	(Alta, Adhesión Tarjeta Crédito)	☐	2.0
(Telefónico, Alta)	☐	0.5	(Telefónico, Adhesión e-commerce)	☐	0.0	(Alta, Adhesión e-commerce)	☐	0.0
(Telefónico, Media)	☐	0.0	(e-mail, Adhesión Tarjeta Débito)	☐	0.3	(Media, Adhesión Tarjeta Débito)	☐	0.0
(Telefónico, Baja)	☐	0.0	(e-mail, Adhesión Tarjeta Crédito)	☐	0.3	(Media, Adhesión Tarjeta Crédito)	☐	0.0
(e-mail, Urgente)	☒	0.0	(e-mail, Adhesión e-commerce)	☐	0.0	(Media, Adhesión e-commerce)	☐	0.0
(e-mail, Alta)	☒	0.0				(Baja, Adhesión Tarjeta Débito)	☐	0.0
(e-mail, Media)	☐	0.0				(Baja, Adhesión Tarjeta Crédito)	☐	0.0

Enseguida nos mostrará los distintos pares existentes para los valores ingresados, tanto para que podamos indicar el peso de cada uno o si queremos remover alguno de esos pares por ser inválidos según las reglas del negocio. En el ejemplo ingresamos mayor valor al peso de los pares que son más importantes para la solicitud de servicios, y removimos los pares (email, Urgente) y (e-mail, Alta), ya que no se permite hacer solicitudes por correo electrónico de prioridad alta y urgente.

Algorithm "prow"

#	Results for a maximum of 36 combinations
1	{Presencial,Urgente,Adhesión Tarjeta Crédito}; sel. factor=8.0
2	{Presencial,Urgente,Adhesión Tarjeta Débito}; sel. factor=8.0
3	{Presencial,Alta,Adhesión Tarjeta Crédito}; sel. factor=6.0
4	{Telefónico,Alta,Adhesión Tarjeta Débito}; sel. factor=3.5
5	{Presencial,Baja,Adhesión Tarjeta Crédito}; sel. factor=3.0
6	{Presencial,Media,Adhesión Tarjeta Débito}; sel. factor=3.0
7	{Presencial,Urgente,Adhesión e-commerce}; sel. factor=3.0
8	{Presencial,Alta,Adhesión e-commerce}; sel. factor=2.0
9	{Telefónico,Baja,Adhesión Tarjeta Crédito}; sel. factor=1.0
10	{Telefónico,Media,Adhesión Tarjeta Crédito}; sel. factor=1.0
11	{Telefónico,Urgente,Adhesión e-commerce}; sel. factor=0.5
12	{e-mail,Baja,Adhesión Tarjeta Débito}; sel. factor=0.3
13	{e-mail,Media,Adhesión Tarjeta Crédito}; sel. factor=0.3
14	{e-mail,Media,Adhesión e-commerce}; sel. factor=0.0
15	{Telefónico,Baja,Adhesión e-commerce}; sel. factor=0.0

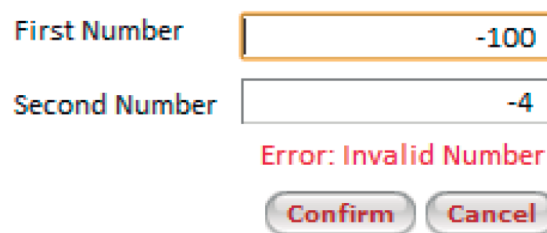
El resultado consta de 15 casos de prueba, los cuales cubren todos los pares válidos, teniendo en cuenta los pesos asignados a cada par. Si decidiéramos hacer producto cartesiano con todos los valores de prueba interesantes tendríamos un total de $3 \times 4 \times 3 = 36$ casos de prueba. O sea, nos estamos ahorrando la ejecución (y en el caso de pruebas automáticas también la automatización) de $36 - 15 = 21$ casos de prueba que, según la teoría de errores considerada, no agregarían valor a nuestro conjunto de casos de prueba, ya que tienen menos probabilidad de encontrar errores.

Para aprovechar la herramienta es muy importante seleccionar bien los valores interesantes a incluir en nuestras pruebas, así como luego determinar qué pares son inválidos. No es lo mismo eliminar casos de prueba que tengan combinaciones inválidas, pues tal vez un mismo caso de prueba cubre varios pares, y si eliminamos uno por tener una combinación inválida, podemos estar a su vez eliminando otra combinación que sea interesante a probar y no se pruebe en ningún otro caso de prueba.

No enmascaramiento de errores

Otra técnica aplicable a la combinación de datos es la de “no enmascaramiento de errores”. Esto implica que en los conjuntos de datos de prueba se utilizará solo un valor de una clase inválida a la vez, pues si se utilizan dos se corre el peligro de no identificar qué valor inválido produce el error.

Ejemplo: una pantalla que tiene dos entradas ambas esperan valores enteros mayores o iguales a cero. Entonces, podemos distinguir como clases inválidas a los números negativos o a las entradas no numéricas. Esta técnica plantea no ingresar en un mismo caso de prueba un valor negativo en ambas entradas al mismo tiempo, pues la idea es ver que la aplicación sea capaz de manejar e indicarle al usuario que el valor de entrada es incorrecto. Si se ingresan valores de clases inválidas tal vez no se logra verificar que para cada entrada incorrecta se hace un manejo adecuado.



First Number

Second Number

Error: Invalid Number

En el ejemplo, imaginen que hay un error al ingresar un valor negativo en el campo correspondiente a “*Second Number*” ya que no se controla correctamente que sea mayor que cero. Entonces, al probar de esta forma estamos “ocultando el error” o “enmascarando el error”.

Una buena forma para utilizar ambas estrategias de combinación de datos que nombramos sería: combinar primero a mano los inválidos según estas consideraciones, y luego con las clases válidas aplicar combinación por pares ya vista.

4. Calcular Valores Esperados

Para cada prueba diseñada, se calculan los valores esperados de acuerdo con las especificaciones del sistema, o a nuestro conocimiento del negocio. En otras palabras, debemos diseñar el oráculo de prueba.

Esta es la única forma de determinar si la aplicación funciona como es esperado o no. Es importante para que al momento de ejecutar las pruebas no se tengan que analizar los resultados, sino que sea posible comparar directamente con lo que ya estaba previsto. Es más, para el caso en que un experto del dominio desee diseñar las pruebas y que alguien más luego sea el encargado de ejecutarlas, dejar esto definido es de vital importancia, porque quizá quien ejecuta las pruebas no tiene total conocimiento de las reglas de negocio, entonces es deseable (conveniente) especificar los valores esperados de antemano.

Con **valores esperados** estamos incluyendo a lo que el sistema muestra en pantalla, archivos que se tengan que modificar, estados de la base de datos, o cualquier otro tipo de acción que deba realizar el software que estamos probando.

Así mismo, una buena práctica es incluir pruebas que deban registrar un fallo, que su resultado esperado sea el manejo de una excepción y su correcto procesamiento. Esto también es conocido como **Testing Negativo**, que consiste en incluir escenarios con aquellas cosas que el sistema debe estar preparado para **no** hacer, o no fallar por tratarse de una situación incorrecta.

Por ejemplo: verificar que, si se intenta hacer una transferencia desde una cuenta bancaria sin saldo suficiente, la operación arroje error y no se restó el monto de la cuenta origen, ni se sumó de la cuenta destino. Esto generalmente queda cubierto al diseñar pruebas con clases inválidas como vimos antes, pero no siempre es así, como, por ejemplo, ver qué pasa si no existe el archivo de configuración, o la tabla de clientes está vacía, o qué ocurre si el sistema se queda sin conexión en la mitad del proceso.

Generalmente se agrega alguna columna a la tabla de casos de prueba, donde se indican las verificaciones por realizar. Podrían ponerse dos columnas, por ejemplo, una para “valores de variables esperados” y otra para “acciones esperadas”. En la segunda columna podrían incluirse cosas como “se despliega mensaje”, o “se agrega registro a la tabla con los datos ingresados”, “se envía un e-mail”.

Atributos de los datos de Prueba

Al identificar los datos de prueba reales, hay cuatro atributos de los datos de prueba que se deben tratar:

Atributo	Descripción	Detalle
Profundidad	grado de variación de los datos de prueba	Profundidad es el volumen o la cantidad de datos que se utiliza para la prueba. La profundidad es una consideración importante, puesto que la escasez de datos puede no reflejar condiciones de la vida real, mientras que el exceso de datos es difícil de gestionar y mantener. Lo ideal es que la prueba se inicie con un pequeño conjunto de datos que ofrezca soporte para los casos de prueba críticos (por lo general, los casos de prueba positivos). A medida que se vaya ganando seguridad durante la prueba, se deben aumentar los datos de prueba hasta que la profundidad

		de datos sea representativa del entorno desplegado (o que sea adecuada y viable).
Ámbito	aplicabilidad de los datos de prueba al objetivo de la prueba	El ámbito es la aplicabilidad de los datos de prueba al objetivo de la prueba, y está relacionado con la profundidad y la extensión. Tener una gran cantidad de datos no significa que sean los datos correctos. Como en el caso de la extensión de los datos de prueba, se debe garantizar que los datos de prueba sean adecuados, es decir, que haya datos de prueba para dar soporte al objetivo de prueba específico.
Extensión	Grado de variación de los valores de los datos de prueba	Se puede aumentar la profundidad de los datos de prueba, simplemente creando más registros. Mientras que, por lo general, es una solución adecuada, no trata las variaciones verdaderas de los datos que se espera encontrar en los datos reales.
Arquitectura	estructura física de los datos de prueba	La estructura física de los datos de prueba sólo es adecuada para cualquier dato preexistente que utilice el destino de la prueba para dar soporte a la prueba, por ejemplo, una tabla de reglas o una base de datos de la aplicación. La prueba no se ejecuta una vez y se finaliza. La prueba se repite en y entre iteraciones. Con el objeto de ejecutar la prueba de modo coherente, seguro y eficaz, los datos de prueba se deben devolver a su estado inicial anterior a la ejecución de la prueba. Esto se aplica, en especial, cuando se va a automatizar la prueba.

Ejemplos:

Extensión: Si estas variaciones en los datos de prueba, es posible que no se puedan identificar defectos (después de todo, no todas las retiradas de los cajeros automáticos son de un importe de 50,00 dólares). Por ello, los valores de datos deben reflejar los valores de datos que se encuentran en el entorno desplegado como, por ejemplo, 10,00 ó 120,00 dólares. Además, los datos de prueba deben reflejar información del mundo real, por ejemplo:

- Nombres que incluyan títulos, valores numéricos, puntuación y sufijos:
 - Dr. James Bandlin, Ms. Susan Smith y Rev. Joseph P. Mayers
 - James Johnson III, Steven Wilshire 3rd y Charles James Ellsworth, Esq.
 - Ellen Jones-Smythe, Brian P. Tellstor
- Direcciones con varias líneas de dirección, por ejemplo:
 - 6500 Broadway Street -Suite 175
 - 1550 Broadway - Piso 17 - Mailstop 75A
- Códigos de ciudad (y país) y números de teléfono que sean reales y mantengan correspondencia:
 - Lexington, MA, EE.UU. + 01 781 676 2400
 - Kista, Suecia +46 8 56 62 82 00
 - París, Francia +33 1 30 12 09 50

Los valores de datos de prueba pueden ser una representación física o una representación estadística de los datos reales para obtener la extensión suficiente. Se sugieren y valoran ambos métodos.

Para crear datos de prueba basados en una representación física de los datos desplegados, identifique los valores (o rangos) permitidos para cada elemento de datos de la base de datos desplegada y asegúrese de que, para cada elemento de datos haya, como mínimo, un registro en los datos de prueba que contenga cada valor permitido.

Por ejemplo:

Ingeniería de Software : Generación de casos y datos de Prueba
Material obligatorio leer para el desarrollo de TP

	Número de cuenta (rango)	Número secreto (entero)	Saldo de cuenta (decimal)	Tipo de cuenta (cadena de caracteres)
	(S) 0812 0000 0000 a 0812 9999 9999 © 0829 0000 0000 a 0829 9999 9999 (X) 0799 0000 0000 a 0799 9999 9999	0000 - 9999	-999.999,99 a 999.999,99	S, C, X
registro 1	0812 0837 0293	8493	-3.123,84	S
registro 2	0812 6493 8355	3558	8.438,53	S
registro 3	0829 7483 0462	0352	673,00	C
registro 4	0799 4896 1893	4896	493.498,49	X

La matriz anterior contiene el número mínimo de registros que representan físicamente los valores de datos aceptables. Para el número de cuenta, hay un registro para cada uno de los tres rangos, todos los números de PIN se encuentran dentro del rango especificado, hay varios saldos de cuenta diferentes, incluido uno que es negativo, y hay registros para cada uno de los diferentes tipos de cuenta. La matriz anterior son los datos mínimos, y la recomendación sería disponer de valores de datos tanto en los límites de cada rango como dentro del rango.

La ventaja de la representación física es que los datos de prueba tienen un tamaño limitado y gestionable, centrado y dirigido a los valores aceptables. Sin embargo, la desventaja es que los datos del mundo real no son totalmente aleatorios. Los datos reales tienden a tener perfiles estadísticos que puede afectar al rendimiento, lo que no se tiene en cuenta al utilizar la representación física.

La representación de los datos de prueba estadísticos son los datos de prueba que reflejan una creación de ejemplos estadísticos (de los mismos porcentajes) de los datos de producción.

Por ejemplo, utilizando los mismos elementos de datos que más arriba, si se analiza la base de datos de producción y se descubre lo siguiente:

- Número total de registros: 294.031
- Número total de tipo de cuenta S: 141.135 (48 % del total)
- Número total de tipo de cuenta C: 144.075 (49 %)
- Número total de tipo de cuenta X: 8.821 (3 %)
- Los números de cuenta y los números de PIN están distribuidos equitativamente los datos de prueba, basados en la creación de ejemplos estadísticos incluiría 294 registros (si se compara con los cuatro que se han indicado más arriba):

	Datos de prueba (al .1 por ciento de producción)	
	Número de registros	Porcentaje
Número total de registros	294	100
Números de cuenta: (S) 0812 0000 0000 a 0812 9999 9999	141	48
Números de cuenta: © 0829 0000 0000 a 0829 9999 9999	144	49
Números de cuenta: (X) 0799 0000 0000 a 0799 9999 9999	9	3

La matriz anterior contiene el número mínimo de registros que representan físicamente los valores de datos aceptables. Para el número de cuenta, hay un registro para cada uno de los tres rangos, todos los números de PIN se encuentran dentro del rango especificado, hay varios saldos de cuenta diferentes, incluido uno que es negativo, y hay registros para cada uno de los diferentes tipos de cuenta. La matriz anterior son los datos mínimos, y la recomendación sería disponer de valores de datos tanto en los límites de cada rango como dentro del rango.

La ventaja de la representación física es que los datos de prueba tienen un tamaño limitado y gestionable, centrado y dirigido a los valores aceptables. Sin embargo, la desventaja es que los datos del mundo real no son totalmente aleatorios. Los datos reales tienden a tener perfiles estadísticos que puede afectar al rendimiento, lo que no se tiene en cuenta al utilizar la representación física.

La representación de los datos de prueba estadísticos son los datos de prueba que reflejan una creación de ejemplos estadísticos (de los mismos porcentajes) de los datos de producción. Por ejemplo, utilizando los mismos elementos de datos que más arriba, si se analiza la base de datos de producción y se descubre lo siguiente:

- Número total de registros: 294.031
- Número total de tipo de cuenta S: 141.135 (48 % del total)
- Número total de tipo de cuenta C: 144.075 (49 %)
- Número total de tipo de cuenta X: 8.821 (3 %)
- Los números de cuenta y los números de PIN están distribuidos equitativamente

los datos de prueba, basados en la creación de ejemplos estadísticos incluiría 294 registros (si se compara con los cuatro que se han indicado más arriba):

La matriz anterior sólo trata los tipos de cuenta. Al desarrollar los datos de prueba más correctos basados en la representación estadística, se deberían incluir los elementos de datos significativos. En el ejemplo anterior, incluiría reflejar los saldos de cuenta reales. Una desventaja de la representación estadística es que no refleja el rango completo de valores aceptables. Por lo general, se utilizan ambos métodos de identificación de datos de prueba para garantizar que los datos de prueba tratan todos los valores y problemas de población / rendimiento.

La extensión de los datos de prueba es adecuada para los datos de prueba que se utilizan como entrada, así como para los datos de prueba que se utilizan para dar soporte a la prueba (en datos

Por ejemplo, en la matriz anterior, los cuatro primeros registros de datos de prueba tratan los valores aceptables para cada elemento de datos. Sin embargo, no hay registros para evaluar los saldos negativos para los tipos de cuenta C y X, por ello, aunque los datos de prueba incluyan de modo correcto saldos negativos (extensión válida), los datos que se sitúan por debajo resultan insuficientes en este ámbito para dar soporte a cualquier prueba que utilice saldos de cuenta negativos para cada tipo de cuenta. Para tratar esta omisión, se necesita ampliar los datos a fin de incluir otros registros, incluidos saldos negativos, para cada uno de los distintos tipos de cuenta.

	Número de cuenta (rango)	Número secreto (entero)	Saldo de cuenta (decimal)	Tipo de cuenta (cadena de caracteres)
	(S) 0812 0000 0000 a 0812 9999 9999 © 0829 0000 0000 a 0829 9999 9999 (X) 0799 0000 0000 a 0799 9999 9999	0000 - 9999	-999.999,99 a 999.999,99	S, C, X
registro 1	0812 0837 0293	8493	-3.123,84	S
registro 2	0812 6493 8355	3558	8.438,53	S

registro 3	0829 7483 0462	0352	673,00	C
registro 4	0799 4896 1893	4896	493.498,49	X
Nuevo registro 1	0829 3491 4927	0352	-995.498,34	C
Nuevo registro 2	0799 6578 9436	4896	-64.913,87	X

El ámbito de los datos de prueba es adecuado para los datos de prueba que se utilizan como entrada, así como para los datos de prueba que se utilizan para dar soporte a la prueba (en datos preexistentes).

Arquitectura: para garantizar la integridad, la seguridad y la eficacia de la prueba, resulta de extrema importancia que los datos de prueba estén libres de todas las influencias externas y que se conozca su estado al principio, durante y al final de la ejecución de la prueba. Para lograr este objetivo de la prueba, se deben tratar dos problemas:

- 🌀 **Inestabilidad/ Segregación:** aislamiento de influencias externas de los datos de prueba
- 🌀 **Estado Inicial:** conocimiento del estado inicial específico de los datos, y posibilidad de volver a dicho estado

Cada uno de estos problemas afecta al modo en que se gestiona la base de datos de prueba, se diseña el modelo de prueba y se interactúa con otros roles.

Inestabilidad / Segregación: los datos de prueba pueden convertirse en inestables por los motivos siguientes:

- influencias externas no relacionadas con la prueba modifican los datos
- los testers desconocen los datos que utilizan los demás testers

Para mantener la seguridad y la integridad de la prueba, los datos de prueba se deben controlar y aislar correctamente de dichas influencias. Las estrategias para asegurar que se han aislado los datos de prueba incluyen:

- **entornos de prueba separados:** los testers tienen su propio entorno de prueba, separado físicamente de los demás. Los testers no comparten nada, es decir, tienen sus propios datos y destino de la prueba. Se puede conseguir, por ejemplo, si cada tester tiene su propio PC.
- **instancias de base de datos de prueba separados:** los testers tienen su propia instancia de datos, aislados de todas las demás influencias. Se comparte el entorno físico, quizá incluso el destino de la prueba, pero si cada tester tiene su propia instancia de datos, se reduce mucho el riesgo de contaminación de los datos de prueba.
- **partición de base de datos / datos de prueba** - todos los testers comparten la base de datos y están bien informados sobre los datos que utilizan los demás (y evitan utilizar los datos de otros testers).
 - Por ejemplo, un tester utiliza los registros 0 a 99 y otro tester utiliza los registros 100 a 199, o bien, alguno utiliza clientes cuyos apellidos empiecen por las letras de la Aa a la Kz, mientras que otro tester utiliza pacientes cuyos nombres empiecen por las letras de la La a la Zz.

Estado inicial: se debe tratar es el del estado inicial de los datos de prueba al iniciar la ejecución de la prueba. En especial, esto se aplica cuando se utiliza la automatización de pruebas. Del mismo modo que el destino de la prueba debe iniciar la ejecución de la prueba en un estado conocido y deseado, lo mismo se aplica a los datos de prueba. Esto contribuye a la condición de repetición y seguridad de que cada ejecución de la prueba sea igual que la anterior.

Para tratar este problema se suelen utilizar cuatro estrategias:

1- Renovación de los datos: para volver los datos de prueba a su estado inicial es la Renovación de datos. Este método implica la creación y el almacenamiento de una copia de la base de datos en su estado inicial. Al terminar la ejecución de la prueba (o antes de la ejecución de la prueba), la copia archivada de la base de datos de prueba se copia en el entorno de prueba para su utilización. De este modo se asegura que el estado inicial de los datos de prueba es el mismo al principio de cada ejecución de la prueba.

Una ventaja de este método es que los datos se pueden archivar en varios estados iniciales diferentes. Por ejemplo, los datos de prueba se pueden archivar en el estado del final del día, el estado del final de la semana o el estado del final del mes, entre otros. De este modo, el tester dispone de un método que le permite renovar rápidamente a un estado determinado con el objeto de dar soporte a una prueba, por ejemplo, al probar los casos de uso de final de mes.

El método que se utiliza depende de varios factores, incluidas las características físicas de la base de datos, la competencia técnica de los testers, la disponibilidad de roles externos (no de prueba) y el destino de la prueba.

2- Reinicialización de los datos: Si los datos no se pueden renovar, el método siguiente más adecuado consiste en restaurar los datos a su estado inicial a través de algún medio programático. La re-inicialización de los datos se basa en herramientas y casos de uso especiales para devolver los datos de prueba a sus valores iniciales.

Debe tener cuidado y asegurarse de que todos los datos, las relaciones y los valores clave se devuelven a su valor inicial adecuado a fin de garantizar que no haya errores en los datos.

Una de las ventajas de este método es que puede ofrecer soporte para la prueba de valores no válidos en la base de datos. En condiciones normales, se debe bloquear la entrada de los valores de datos no válidos en los datos (por ejemplo, por medio de una regla de validación en el cliente). Sin embargo, otro actor puede afectar a los datos (por ejemplo, una actualización electrónica de otro sistema). La prueba debe verificar que se identifiquen y manejen correctamente los datos no válidos, con independencia de cómo se lleve a cabo.

3- Restablecimiento de los datos: "revertir los cambios" realizados en los datos durante la prueba. Este método se basa en la utilización del destino de la prueba para entrar entradas de reversión, es decir, añadir registros/valores que se han suprimido, deshacer las modificaciones de los registros/valores modificados y suprimir los datos que se han añadido.

No obstante, **este método tiene riesgos asociados**, entre los que se incluyen:

- se deben revertir todas las acciones, no sólo algunas

- se basa en casos de uso en el destino de la prueba (que se deben probar para verificar si funciona correctamente antes de utilizarlos para el restablecimiento de los datos).
- las claves de base de datos, los índices y los puntos no se deben o no se pueden restablecer

Si éste es el único método disponible en el entorno de verificación, evite utilizar claves de base de datos, índices y punteros como destinos principales de la verificación. Es decir, por ejemplo, utilice el campo Nombre del paciente para determinar si el paciente se ha añadido a la base de datos, en lugar de utilizar un número de ID de paciente generado por el sistema.

4- Rehacer datos: es el método menos adecuado para tratar el estado inicial de los datos de prueba. De hecho, no se aplica realmente al problema. En vez de ello, al terminar la ejecución de la prueba el estado de los datos se convierte en el nuevo estado inicial de los datos de prueba. Por lo general, **requiere modificar los datos de prueba utilizados para la entrada, y los casos de prueba y los datos de prueba utilizados para la evaluación de los resultados.**

Hay algunas instancias en las que es necesario, por ejemplo, al final del mes. Si no existen archivados los datos antes del final del mes, se deben ejecutar los datos de prueba y los scripts de prueba de cada día y semana con el objeto de "rehacer" los datos para el estado que se necesita para la prueba del proceso de fin de mes.

Entre los riesgos asociados a este método se incluyen:

- las claves de base de datos, los índices y los puntos no se pueden restablecer (y no se pueden utilizar para la verificación)
- los datos cambian constantemente
- requiere esfuerzo adicional para certificar la verificación de los resultados

Ver Video: <https://www.youtube.com/watch?v=SetmzVB1cEQ> explica como usar la herramienta que sigue, mockeando o inventando datos.

Otra herramienta : **Mockaroo. Generador de datos para pruebas.**



Mockaroo es una gran herramienta para los testers. Se trata de una herramienta web desarrollada por **Mark Brocato**, desde la que vamos a poder generar hasta 100.000 líneas de datos realistas para pruebas que podremos exportar en formato CSV, Tab-Delimited, JSON, SQL, Excel y DBUnit XML.

En muchas de nuestras pruebas, por ejemplo con jmeter, tener ficheros csv con datos de prueba realistas es de una inestimable ayuda. Y tenerlos con la rapidez y calidad que nos da mockaroo es todavía mejor. Además, probar con datos realistas hará que las pruebas de la aplicación sean más robustas, puesto que vamos a poder detectar errores que podrían producirse en producción con datos reales.

Hasta ahora, una posible opción era generar estos archivos con herramientas como Microsoft Excel u Open Office Calc, con las que copiando y arrastrando se podían generar archivos interesantes en un tiempo aceptable.

Ingeniería de Software : Generación de casos y datos de Prueba

Material obligatorio leer para el desarrollo de TP

Pero **mockaroo** va más allá. Nos va a permitir indicar los campos que queremos que tenga nuestro archivo, tanto el nombre, como el tipo de campo que queremos que sea. Hay infinidad de tipos de campos, no desde un punto de vista de desarrollo (string, integer, boolean, ...) sino desde un tipo de vista funcional: nombre, apellido, ciudad, tarjeta de crédito (diferenciando entre visa, mastercard o american express), código de país, color, talla, hora, teléfono, tipo MIME.

The image shows the Mockaroo website interface. The top section is a form to configure fields for data generation. It has three columns: Field Name, Type, and Options. Fields include id (Row Number), first_name (First Name), last_name (Last Name), email (Email Address), country (Country), and ip_address (IP Address v4). Each field has a 'blank' checkbox and a percentage input. Below the fields is an 'Add another field' button. At the bottom of the form, there are settings for '# Rows' (1000), 'Format' (CSV), and a checked 'include header' checkbox, followed by a 'Generate Data' button.

The bottom section shows a preview of the generated data under the heading 'Choose a Type'. It features a filter input and tabs for different categories: All (84), Basic (22), IT (10), Personal (17), Location (14), Money (7), Health (10), and Advanced (4). The data is displayed in a grid with three columns. The first column shows Bitcoin addresses, City (New York, Berlin, London), Country (Germany, France, Japan), Credit Card Type (visa, mastercard, americanexpress), and Custom List. The second column shows Blank (Always generates a null value), Color (Red, Blue, Black), Country Code (ES, GR, FR), Currency (Dollar, Euro, Peso), and Date (07/04/2013, 4.7.2013, 04-Jul-2013). The third column shows Boolean (true, false), Company Name (Google, Home Depot, General Electric), Credit Card # (4017959045824, 5349690971837932, 372301294706546), Currency Code (USD, EUR, MXN), and Domain Name (google.com, wikipedia.org, nih.gov). At the bottom of the preview, there is a message: 'Can't find what you're looking for? You can mock almost anything using Regular Expressions...' and a 'Close' button.

Otras herramientas:

- [GenerateData](#)
- [DBMonster](#): para generar datos aleatorios en la base de datos. Muy completo pero algo más complejo.
- [CSV Data Generator](#): basado en Ruby genera ficheros CSV.
- [Datagenerator](#): Permite introducir datos en Mysql, Firebird, Interbase, MSSQL, Oracle, SQLite y PostgreSQL.

Ing. Mónica Colombo